



Degree project in Computer Science and Engineering

Second cycle, 30 credits

Selecting a service mesh implementation for managing microservices

ASAWARI JOSHI

Selecting a service mesh implementation for managing microservices

ASAWARI JOSHI

Master's Programme, ICT Innovation, 120 credits

Date: October 17, 2022

Supervisors: Jaume Anguera Peris, Erfan Yousefi

Examiner: Viktoria Fodor

School of Electrical Engineering and Computer Science

Host company: Infor(Sweden) AB

Swedish title: Välja en service mesh-implementering för hantering
av mikrotjänster

Swedish subtitle: En jämförelse mellan service mesh-teknologier
för valda parametrar

Abstract

Microservice architectures are the base of modern cloud applications. With the adoption of microservices, application teams manage to reduce codebase complexity and write more modular services that can run inter-dependently. Despite all the advantages offered by microservices over monolithic architectures, they introduce additional complexities, such as handling inter-service communication, ensuring security, introducing traceability, and achieving acceptable performance. To manage all these microservices, the orchestrator tool Kubernetes has shown promising results, but it is not sufficient to tackle all the cross-cutting concerns of the applications. With more complex architectures, it becomes difficult to troubleshoot and trace inter-service application programming interface calls. For this purpose, the service mesh technology emerged and is being introduced in many companies.

There are different implementations available for service mesh, with some being more compatible with specific cloud providers than others. The selection of an adequate service mesh shapes the entire application deployment process. If chosen wrongly, it might introduce further complexities or performance losses. This thesis investigates ways to make this decision more efficiently. It takes into account the company's needs of service mesh and compares implementations to meet performance expectations with minimal deployment efforts. Specially, we perform different testing scenarios to compare AWS cloud specific, AWS App Mesh with most widely used, Istio service mesh. The outcome suggests that AWS App Mesh performs consistently for all cases with acceptable latency ranges, but, Istio outperforms AWS App Mesh under standard conditions, such as high but manageable request load on the application. In addition, Istio can achieve the same deployment with less lines of code as compared to that of AWS App Mesh. On the other hand, resource utilization by these service meshes is found to be a non-conclusive factor for selecting the service mesh implementation. With these outcomes, application teams can make more well-informed decisions to run the production-grade workloads with an efficient service mesh implementation.

Keywords

Service mesh, Istio, AWS App Mesh, Performance testing, Fortio, Latency distribution, Microservice architecture

Abstract

Sammanfattning

Mikroservicearkitekturer är basen för moderna containerapplikationer. Med antagandet

av mikrotjänster lyckas applikationsteam minska kodbaskomplexiteten och skriva fler modulära tjänster som kan köras beroende av varandra. Trots alla fördelar som mikrotjänster erbjuder framför monolitiska arkitekturer, introducerar de ytterligare komplexitet, såsom kommunikation mellan tjänster, säkerhet, spårbarhet och prestanda. För att hantera alla dessa mikrotjänster har orkestreringsverktyget Kubernetes visat lovande resultat, men det är inte tillräckligt för att ta itu med alla övergripande problem med applikationerna. Med mer komplexa arkitekturer blir det svårt att felsöka och få synlighet i mikrotjänster. För detta ändamål växte servicemesh-tekniken fram och introduceras i många företag.

Det finns olika implementeringar tillgängliga för servicemesh, där vissa är mer kompatibla med specifika molnleverantörer än andra. Valet av ett adekvat servicenät formar hela applikationsdistributionsprocessen. Om det väljs fel kan det leda till ytterligare komplexitet eller prestandaförluster. Denna avhandling undersöker sätt att göra detta beslut mer effektivt. Den tar hänsyn till företagets behov av servicenät och jämför implementeringar för att möta prestandaförväntningar med minimala implementeringsinsatser. Speciellt genomför vi olika scenarier för prestandatestning för att jämföra AWS App Mesh med Istio servicemesh. Resultatet tyder på att AWS App Mesh presterar konsekvent för alla fall med acceptabla latensintervall, men Istio överträffar AWS App Mesh under standardförhållanden, såsom hög men hanterbar förfrågningsbelastning på applikationen. Istio kan uppnå samma distribution med färre rader kod jämfört med AWS App Mesh. Resursutnyttjande av dessa tjänstenät har visat sig vara en icke avgörande faktor för val av tjänstenätimplementering. Med dessa utfall kan företaget där detta examensarbete genomförs, Infor (Sweden) AB, fatta mer välinformerade beslut för att driva de produktionsmässiga arbetsbelastningarna med en effektiv implementering av servicenät.

Nyckelord

service mesh, Istio, AWS App Mesh, Performance testing, Fortio, Latency distribution, Microservice architecture

Acknowledgments

I would first like to thank my industrial supervisor at Infor, where I conducted my research, Erfan Yousefi. He helped me understand the need of this research and also contributed to resolving several configuration and coding issues. He welcomed me in his team at the company, and supported me throughout the research process. I would also like to thank my thesis supervisor, Jaume Anguera Peris at KTH for guiding me to design and structure the research. He tremendously helped in organizing the contents of the thesis and answered all my queries with helpful insights. I am thankful for all the help I received from my team at Infor during my research process. Furthermore, I would like to thank my examiner, Professor Viktoria Fodor for helping me understand the selection criteria for the project and giving her valuable feedback from time to time.

Stockholm, October 2022

Asawari Joshi

Acknowledgments

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem	3
1.3	Purpose	4
1.4	Benefits, Ethics and Sustainability	4
1.5	Research Methodology	5
1.6	Delimitations	6
1.7	Structure of the thesis	6
2	Background	9
2.1	Microservice architecture	9
2.1.1	From monolithic to microservices	9
2.1.2	Containers and the need for orchestration	12
2.2	Kubernetes	13
2.2.1	Kubernetes Concepts	13
2.2.2	Kubernetes Components	14
2.3	Service Mesh emergence	16
2.3.1	Challenges in Microservices	16
2.3.2	Service Mesh	17
2.4	Selection of service mesh implementations	22
2.4.1	Meshes	22
2.4.2	Selection Criteria	23
2.5	Istio	24
2.5.1	Architecture	24
2.5.2	Core functionalities	26
2.6	AWS App Mesh	28
2.6.1	Architecture	28
2.6.2	Available and selected features	30

2.7	Related work	31
2.7.1	Performance benchmark and resource utilization	32
3	Method and implementation	41
3.1	Selection of tools and technologies	41
3.2	Application and cluster details	43
3.3	Application deployment on Istio and AWS App Mesh	44
3.4	Performance testing	50
4	Results and Analysis	53
5	Discussion	65
6	Conclusions and Future work	69
6.1	Conclusions	69
6.2	Limitations	70
6.3	Future work	70
6.4	Reflections	71
A	Selection criteria	83
B	Sample cost calculations for service mesh solutions	85
C	Listings	87

List of Figures

2.1.1 Process to move from a monolith to a microservice architecture [1, 2]	12
2.2.1 Kubernetes architecture	15
2.3.1 Different Service mesh implementations [3]	19
2.3.2 Service Mesh Architecture [4, 2]	20
2.5.1 Istio service mesh architecture [5, 6]	27
2.6.1 AWS App mesh components(CRDs) [7, 8]	30
3.2.1 Application without any mesh [9]	44
3.3.1 Application Bookinfo with Istio service mesh	48
3.3.2 AWS App Mesh: Service reviews-v1 architecture [10]	49
3.3.3 Application Bookinfo with AWS App Mesh [10]	49
4.0.1 Istio and AWS App Mesh: Pre-benchmarking test	55
4.0.2 Maximum QPS supported by Istio and AWS App Mesh	56
4.0.3 Istio: Latency distribution graph when QPS is 0	57
4.0.4 AWS App Mesh: Latency distribution graph when QPS is 0	57
4.0.5 Actual QPS values supported by Istio and AWS App Mesh when QPS is 743	58
4.0.6 Istio: Latency when QPS = 743, number of connections = 32 and the test running time is 5 minutes per sample (total 30 samples collected)	59
4.0.7 AWS App Mesh: Latency when QPS = 743, number of connections = 32 and the test is run for 5 minutes per samples (total 30 samples collected)	59
4.0.8 Data plane: CPU	61
4.0.9 Control plane: CPU	62
4.0.10 Data plane: Memory	62
4.0.11 Control plane: Memory	63

List of Tables

2.4.1 Support for features per service mesh implementation	25
2.7.1 Statistics collected from graph approximations found in [11]: Latency (P99) in milliseconds, CPU in millicores, and Memory in GB	33
2.7.2 Logarithmic latency comparison	35
2.7.3 Test data for different service mesh implementations. The number of connections for each test is 32.	36
2.7.4 Comparison of latency-related statistics, setup details, and test scenario details collected from references	37
2.7.5 Comparison of CPU and memory statistics collected from [12, 13]	39
3.1.1 Selected tools and technologies	42
3.2.1 Cluster and application set up details	45
4.0.1 Initial test data for Istio and AWS App Mesh	54
4.0.2 Comparison based on gathered data and analysis	63

List of acronyms and abbreviations

AWS Amazon Web Services

CI/CD Continuous Integration/Continuous Deployment

CRD Custom Resource Definitions

CRE container runtime engine

EKS Elastic Kubernetes Service

GCP Google Cloud Platform

JWT Java Web Token

NLB Network LoadBalancer

QPS Queries Per Second

RPS Requests Per Second

VPC Virtual Private Cloud

List of acronyms and abbreviations

Chapter 1

Introduction

This section explores the idea and the necessity of a service mesh in detail. It also conducts a comparative research to identify the most suitable implementation of service mesh technology for production-grade environments.

1.1 Background

In recent years, companies have shown a major tendency towards cloud computing. The move demands changes in the way applications are designed. With infrastructural changes, the deployed application needs to utilize the benefits that the cloud provides. That means, essentially developing a strategy to redesign the legacy and complex codebases into manageable and smaller deployment units. In modern terms, it is important to move from monolithic architectures to microservice architectures to fully adopt a cloud-native approach wherever the decomposition of existing applications is possible.

Monolithic Architectures are frameworks that combine all functions of an application in a single application unit that is easy to develop, test, deploy and scale [14]. Due to this simplistic nature, it provides low architectural complexity, although it reduces the application's reusability and renders a large and complex codebase. To mitigate these limitations, Microservices Architectures leverage a service-oriented architecture style to support applications that contain a collection of loosely coupled software

services [14]. Microservices are distributed in nature and allow multiple components to be developed and deployed independently, allowing code reusability, faster release cycles, and a maintainable codebase. Docker and Kubernetes are the frameworks that leverage microservice architectures to reduce the gap between the development and deployment process of complex applications. They deliver microservices in a fast and efficient manner by packaging code in readily deployable service units.

The distributed nature of microservices leads us to reflect on the idealistic assumption that the underlying network connecting those services is reliable, highly available, has infinite bandwidth, and exhibits no data loss. Kubernetes provides several features to ensure application reliability and high availability. It also offers bare minimum support for inter-service communication along with service discovery. Yet, despite these in-built features and mechanisms, there are additional requirements commonly requested by development and/or deployment teams, such as:

1. Provide effective, fast, secure, available, and reliable communication between services distributed across the cluster.
2. Monitor and capture every message exchanged between services.
3. Measure the health of service communication requests and collect low-level metrics.
4. Achieve more visibility into system communication.

In a production-grade environment, the number of microservices could be enormous, and in case of system failure or performance problems, it is difficult to isolate the cause of the problem or find a particular microservice slowing down the system. Hence, we need a new layer managing inter-service communication called service mesh.

A service mesh provides observability, security, and resilience in the application architecture. It also provides the logic to monitor, control, and manage not only microservices but the underlying infrastructure by intercepting service requests. Service mesh supports a network-policy-driven approach, allowing controlled traffic flow based on policy definitions in face of constantly changing network topologies. Moreover, it makes the development process easy and fast as it does not require the developer to focus on inter-service communication, circuit breaking, and timeout implementation at the application level.

1.2 Problem

This thesis focuses on driving cloud-native initiatives to move from monolithic to microservice-based architectures. To support the production-grade deployment of microservices, Kubernetes is being used as the orchestration tool. As the legacy codebase is large and complex, it is essential to move to the microservices slowly and efficiently. Kubernetes alone is not sufficient to promise all the cross-cutting concerns such as routing, encryption, security, and metrics collection. Therefore, Information Technology firms are choosing service mesh technologies to ensure reliability, security, and observability regarding the microservices deployed in production. A service mesh has a variety of open source and cloud-specific implementations, so it is important to understand the organization-specific use-cases and when choosing a suitable service mesh. In this study, performance is considered the main factor when it comes to selecting the service mesh.

Istio, Linkerd, and Consul are the most commonly used open-source service meshes. Istio is used by leading companies in the IT industry like Google, IBM, and Lyft, whereas Linkerd is backed by CNCF. Istio is more configurable and provides a rich set of features at the cost of higher response times. For example, features such as circuit breaking, delay injections, and mesh expansion are not supported in Linkerd. Linkerd provides better performance in terms of response times and resource utilization as it uses a lightweight linkerd2-proxy. Consul is simple and is expected to be similar to Linkerd in terms of performance, but does not have preconfigured integrations with open-source monitoring and observability tools such as Prometheus and Grafana. Overall, Istio stood out as the logical choice for it being configurable, open-source, and well-supported.

As per the official service mesh documentation, selecting an app mesh has long-term effects on the application deployment, and it is important to compare different implementations, keeping an essential feature-set for a specific use case in mind. At the time of writing, Amazon Web Services (AWS) is the base of all the services provided to the customers at Infor. Hence, a service mesh well integrated with AWS infrastructure, known as App Mesh, is a valid candidate. Therefore, we decided to focus on comparing AWS App Mesh and Istio for performance, ease of implementation, and cost-efficiency.

The three sub-questions mentioned in the SQs below need to be answered to conclusively answer the main research question RQ1:

- RQ1: Which service mesh technology is better suited for Infor's

production-grade microservice architecture?

- SQ1: Which service mesh candidate is easier to configure and maintain?
- SQ2: Which service mesh candidate provides better performance in terms of response latency and request throughput?
- SQ3: Which service mesh candidate consumes lower resources with the possibility to maximize resource utilization?
- SQ4: What are the cost implications of service mesh candidates?

1.3 Purpose

The purpose of this project is to identify the implementation of service mesh that would be most useful for production-grade environments, involving Kubernetes as the orchestration tool for container management. The objective of the thesis is to analyse AWS App Mesh and Istio for their implementation efforts, performance, and resource utilization. The decision of this choice is made based on a literature survey, implementation overview, and an analytical comparison between the performance of the sample application deployed using the two meshes. The research will also gain some insights into the monitoring solutions provided by both the meshes.

1.4 Benefits, Ethics and Sustainability

Data centers are considered the backbone of entire data-providing networks in cloud platforms, but they are responsible for roughly 1% of global electricity consumption [15]. Leading cloud providers such as Google Cloud Platform (GCP), AWS, and Azure plan to have carbon-neutral energy running data centres in the coming 5-10 years [16]. As the implementation in the thesis mainly involves the AWS cloud, the carbon footprint implication of the selected cloud is worth mentioning. As per the official AWS documentation [17], it is 88% more energy-efficient than the on-premises data centres.

Virtualization in data centres causes higher energy consumption as compared to non-virtualized environments. In a Kubernetes cluster with service mesh implementation enabled, the number of resources running in the virtualized environment almost doubles, resulting in more resource

consumption than in a cluster without any service mesh. On the other hand, if the service mesh is implemented, it might even save energy because network function libraries in the business logic code would be unnecessary. Thus, from a sustainability point of view, it is important to analyse the capabilities provided by different service mesh architectures to save energy consumption in the implementation.

In the AWS App Mesh implementation, the control plane does not consume as many computing resources as in the Istio mesh. Additionally, in AWS App Mesh, the control plane can be launched in a serverless service, known as AWS Fargate, where the deployed containers do not require dedicated resources. This can reduce power consumption, as resources shall be used only when necessary and not be dedicated or reserved.

From an ethics perspective, data about Infor's product or internal architectural decisions have not been exposed in the research. The research implementation would include open source tools and technologies such as Istio, Prometheus, Grafana, and locust as well as App Mesh, a paid service provided by AWS, paid by the organization. Data collected from the paid service is not deemed confidential and hence can be used for competitive analysis.

1.5 Research Methodology

This research involves both qualitative and quantitative methodologies for the comparative analysis of service mesh implementations. Qualitative research deals with understanding implementation efforts for deploying an application to selected service meshes. For example, monitoring the deployed application with the help of service mesh, helps us understand which service mesh provides better visibility and traceability into the application. An exploratory approach is conducted during performance testing on selected technologies, as there are a variety of tools and technologies available for conducting such type of testing. Parameters such as success rate, response time, request throughput, and resource utilization have been chosen for quantitative analysis. Relevant data is collected using Prometheus and plotted for more readability with Grafana. The research refers to not only books and research papers but also online forums, official websites of relevant technologies as well as YouTube channels. Analytical research has been conducted to compare the costs involved in deployment across chosen service meshes. The research has been conducted with Infor's production-grade application needs in mind. A

sample microservice-based application is selected, following the microservice architecture model Infor uses for some of its applications. The choice of cloud is made based on the current infrastructural choices made by the organisation. The chosen application is then deployed on identical Kubernetes clusters in the AWS cloud, with one cluster enabled with Istio mesh while the other, with AWS App Mesh. Implementation efforts are subjective, as familiarity with a specific technology makes implementation easier than non-familiar technologies. Keeping this in mind, efforts are compared only concerning the number of steps required to perform certain operations during deployments. For example, a certain service mesh functionality can be enabled with a single change in a file in one implementation, whereas the other needs more changes. The architectural details and implementation theories are explained in relevant sections for the chosen service meshes.

1.6 Delimitations

The thesis involves a competitive analysis of Istio and AWS App Mesh based on selected parameters deemed important for deployment considerations. It does not include all features supported by these technologies. Instead, only a set of parameters have been chosen for the comparison, and the research may not contribute to the selection decision if the parameters are changed. For example, mechanisms for implementing retry and timeout logic in the mesh are not compared, whereas performance metrics such as application throughput, error rates, CPU, and Memory resource utilization have been compared to make a conclusive decision in the research. The aim of the research is not to prove one technology better than the other in all aspects, but only in a limited number of aspects and selected scenarios. The research uses a sample application for performance testing, and the numbers could be different for other applications or other scenarios. Moreover, the project does not include a deep dive into all the service mesh implementations due to time constraints. The selected meshes have been picked based on the company's infrastructural choices and the capabilities of the meshes.

1.7 Structure of the thesis

Chapter 1 gives an overview of the thesis, including the motivation and brief background to set the required context for the following sections.

Chapter 2 includes a detailed understanding of microservice architecture and container orchestration, followed by an introduction to selected service-mesh technologies for the thesis. It also discusses existing works related to the service mesh and the performance benchmarking area. Chapter 3 gives a brief idea of chosen tools and technologies, as well as a description of the implementation details for selected service meshes. Later, it highlights the application deployment and procedure to conduct performance testing on selected service meshes. It then briefs about the cost implications of selected technologies. Chapter 4 describes the results drawn from the analysis of gathered data and qualitative aspects such as implementation efforts and traceability of the deployed application. Chapter 5 summarizes different concepts derived from literature research and conducted experiments. In Chapter 6, all the important findings of the thesis are gathered, and a possible future work is suggested based on the identified limitations.

Chapter 2

Background

2.1 Microservice architecture

To understand the microservice architecture, it is important to study the traditional application architecture, that is, monolith. Both architectures have benefits and limitations, although, microservices is considered a modern architecture that overcomes major limitations present in the monolithic architecture. In either case, selecting which architecture to implement is a vital decision for application's lifecycle. The following sections discuss the fundamentals of each architecture in detail.

2.1.1 From monolithic to microservices

The traditional approach to design applications involved monolithic architectures. Monolith architectures demand designing, developing, and deploying an entire application as a single unit. Specially, all components of the application such as the database, business logic, background processes, and remaining functional components are developed in a single deployable unit [18, 19]. Thus, software as a whole is delivered in one single application. This architecture has some advantages, such as simplification of developer workflow, deployment architecture, troubleshooting, and monitoring. Monolithic architecture makes code reuse an easy and simple decision, as well as being traditional, more tried, and tested [20, 21]. Therefore, it is more trusted than the newer architectural patterns [21].

As teams grow in size, the disadvantages of monolith architectures begin to surface. Components in a monolith are tightly coupled, making it hard to change one component without causing failures in another. Monoliths have large codebases, which are challenging to comprehend for newcomers to the team. Before making a small change related to a specific component in the codebase, the newcomers need to go through all the components in detail to avoid causing failures. It is difficult to draw boundaries between components in a monolith, hence hard to delegate ownership to individuals in the team. It does not follow the single responsibility principle. The problem described as delivery contention gives rise to confusion about the ownership of specific lines of code in the monolith and slows down the decision-making process [22]. For every small change, the entire application needs to be redeployed, which causes more deployments and restart times [21, 23]. As the impact of a small change is not fully understood, it leads to enormous amounts of manual testing. A small bug in a single component can bring down the entire application and can cause unavailability in the application until the bug is fixed, making the application unreliable. Scaling is a challenge in monoliths, as different components may have conflicting resource demands. Continuous and faster release cycles are hard to achieve, and adapting to new languages or development frameworks is an expensive and time-consuming decision [21].

On the contrary, microservice architectures divide an application into a collection of highly maintainable, testable, loosely-coupled, and independently deployable services [1]. They are modelled around business capabilities and communicate with one another via Application Programming Interfaces (APIs) or network endpoints [20, 21]. Figure 2.1.1 shows the transition from monolith architectures to microservice architectures.

Microservice architectures have a variety of advantages. Application components are robust, independent, and consequently scalable, as they are considered individual microservices. For developers and quality analysts, it is easy to understand the relevant microservices rather than the entire application codebase. This results in newcomers spending less time in onboarding activities. Microservices follow the single responsibility principle, making a clear separation of concerns for individual developers or small teams owning a specific microservice. An individual microservice-specific change causes redeployment of the concerned microservice and not the entire application, thus ensuring partial or complete availability. As microservices are independent of one another, they can be redeployed and scaled easily. It is easy to isolate faults, making microservices reliable. Microservices architectures are language or framework independent. This

allows developers or teams to use different technology stacks per component in the application. Continuous deployment and automation are promoted in microservice architectures.

Despite all the mentioned advantages of microservices over monolith architectures, microservices can introduce added complexities to the application. As microservices work essentially in a distributed system, they need to take into account the commonly described fallacies of the distributed systems [20, 24]. They need to implement an inter-process communication mechanism and also address the latencies, retry, and timeout mechanisms in case of API call failures. They also need to ensure the availability and reliability of the underlying network. Testing strategies are complex, as dependencies of the microservices need to be launched to ensure integrity between them. Deployment, scaling, and monitoring bring additional complexities, such as dependency management, service discovery mechanism, and defining trigger points for scaling automation [25].

Adapting to modern microservice-based architecture is not an easy decision, as it highly affects the development, testing, and release process of the application. It is important to identify and ensure the need to migrate from monolith to microservice architecture. If certain needs are not fulfilled in the monolithic architecture, moving to microservices would be a viable option. For a small team owning an entire application, it may not be a wise decision to drop the current architecture for microservices, as it is a financially and time-wise costly decision to restructure the application with limited resources. Whereas, for largely growing teams, microservices could be the most reliable choice for moving applications to the cloud. Improving robustness, scaling for increasing customer demands, introducing a new programming language for a component, and decreasing system downtime can be possible reasons to consider moving to a microservice architecture [20].

Infor is a leading brand in Enterprise Resource Planning(ERP) systems and has been providing managed services to its customers for more than 20 years with on-premises systems. In recent years, Infor has moved its infrastructure to the cloud to provide more reliable and efficient service with faster updates. This change goes hand-in-hand with restructuring the existing monolith applications to microservice architecture to fully utilize the cloud platform.

Background

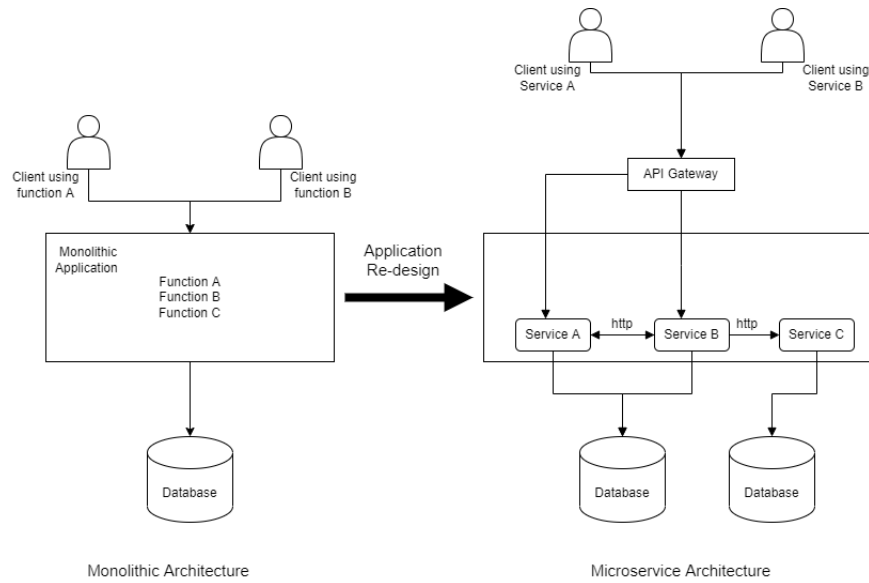


Figure 2.1.1: Process to move from a monolith to a microservice architecture [1, 2]

2.1.2 Containers and the need for orchestration

Virtualization is a backbone of cloud computing technology and modern cloud platforms such as Azure, GCP, and AWS are reliant on this technology [26, 27]. Virtual machines allow application isolation, efficient resource utilization, and increased application availability. Despite the advantages of virtual machines over physical servers, they can cause resource overhead [28].

A container is a lightweight virtualized process that requires less startup time, thereby making it more resource-efficient, easily portable, and highly scalable [29, 30]. It is a new, and more modern form of virtualization, called operating-system-level virtualization, as it divides user space to isolate multiple processes [31]. By doing so, it optimizes resource consumption on a single host machine. A microservice defines the design of software, whereas a container defines the packaging of the software. A Container can run a microservice with all dependencies, such as libraries, executables, configuration files, and binaries packed inside [29]. To utilize both microservices, and container architectures, it is beneficial to use them in combination with each other.

A host machine requires a container runtime engine (CRE) to run containers. Docker is a widely used open-source CRE that allows the development of microservices in portable LXC containers to automate faster

deployments [32, 33]. It runs each container in a separate namespace, isolating it from the underlying operating system, thereby enabling multi-tenancy between applications [32]. Docker manages the container designing and deployment process, but it does not have the central management capabilities required to administer multi-container applications. For instance, if an application has multiple interdependent microservices running as containers, a central system is essential to handle the communication, scaling, and availability of microservices [34].

2.2 Kubernetes

Kubernetes is a container orchestration tool, originally designed by Google, to allow faster, modular, and efficient development of applications [35]. Its main responsibility is to orchestrate containers by managing clusters. Specifically, it places containers on the available machines in the cluster to optimize resource utilization, and it replaces unhealthy or dead containers to provide high availability. It also allows service discovery to communicate with other microservices running in the cluster. Kubernetes is compatible and can be easily integrated with modern cloud platforms, such as AWS, GCP, and Azure [28, 35]. To understand the advantages and limitations of Kubernetes, it is essential to deep dive into the technology, its components, and relevant architectures. Figure 2.2.1 depicts Kubernetes architecture in detail.

2.2.1 Kubernetes Concepts

This section discusses important components of Kubernetes. Other components such as volumes, stateful, secrets, etc. have not been included as they are not relevant to the scope of this thesis.

- A **cluster** is a collection of hosts, networks, and storage resources required to run Kubernetes. The entire architecture may contain one or multiple clusters. A cluster has one or several masters and worker nodes to run application workloads.
- The **master node** comprises the control plane. The control plane has three components, namely, the scheduler, the API-server, and the controller manager. The master node maintains the placement, availability, and events of the pods. There can be multiple masters in the cluster to make it highly available.

- The **worker node** makes up the data plane. It includes Kubelet and Kube-proxy components. It runs pods and is controlled by the master node(s) [36].
- A **pod** is the smallest deployable artefact and contains one or multiple containers. A single microservice is deployed in a single container. Containers within a pod share the same network namespace(IP address and port space) and communicate with each other over standard inter-process communication protocol or localhost [35].
- A **replica** is the number of pods of a specific microservice running in the cluster. Replica controllers or replica sets promise to always have a specific number of pods running in the cluster. An unhealthy or crashed container is identified and replaced to maintain the replicas.
- A **Service** represents a set of pods to expose functionality or a microservice to users. They have different types to control the access to internal resources from outside entities and/or other microservices. They can be load-balanced by Kubernetes or developers themselves.
- A **namespace** is a virtual cluster. A single cluster can have multiple namespaces. Microservices across namespaces can communicate only via public interfaces. It provides better separation between multiple stacks of microservices deployed on a single cluster.

2.2.2 Kubernetes Components

As discussed earlier, the master and worker nodes in a cluster have several components that conduct the core functionalities of container orchestration.

The master is responsible for planning cluster operations, managing the cluster state, monitoring the nodes and containers, and scheduling pods on nodes. Information about pods, such as the time of deployment, pod-node association, and cluster state, is stored on a highly available key-value store known as the Etcd cluster. The scheduler identifies the right node to place the pods on based on the capacity of the worker node, the resource requirements of the container, several constraints such as taints and toleration, and node-affinity rules. There are different types of controller-managers available on the cluster for various purposes. Node-controllers manage the on-boarding of new nodes, handling unhealthy, not-ready, or destroyed nodes. The replication controller ensures that the specified number of containers is always running in

Background

the cluster by replacing a terminated pod or adding/removing replicas to match the desired replication count. Kube-API server is the primary management component of the cluster. It is responsible for orchestrating all operations in the cluster, and it is exposed to external users and systems to perform management operations on the cluster.

Every worker node has a component called Kubelet that is responsible for conducting actual workloads assigned to it by the master. It registers the node to the cluster and gathers information about the node it runs on, and the status of the pods running on the same node. The Kube-API server fetches continuous reports about the gathered information to keep the Master node updated with the latest resource and workload-related information. Microservices running across different Worker nodes need to communicate with each other. A component called Kube-proxy service manages this communication by enforcing certain access rules to reach one node from another. All these components can be hosted as containers in the Kubernetes ecosystem. To support running containers, Kubernetes requires a CRE such as Docker, Containerd, or Rkt.

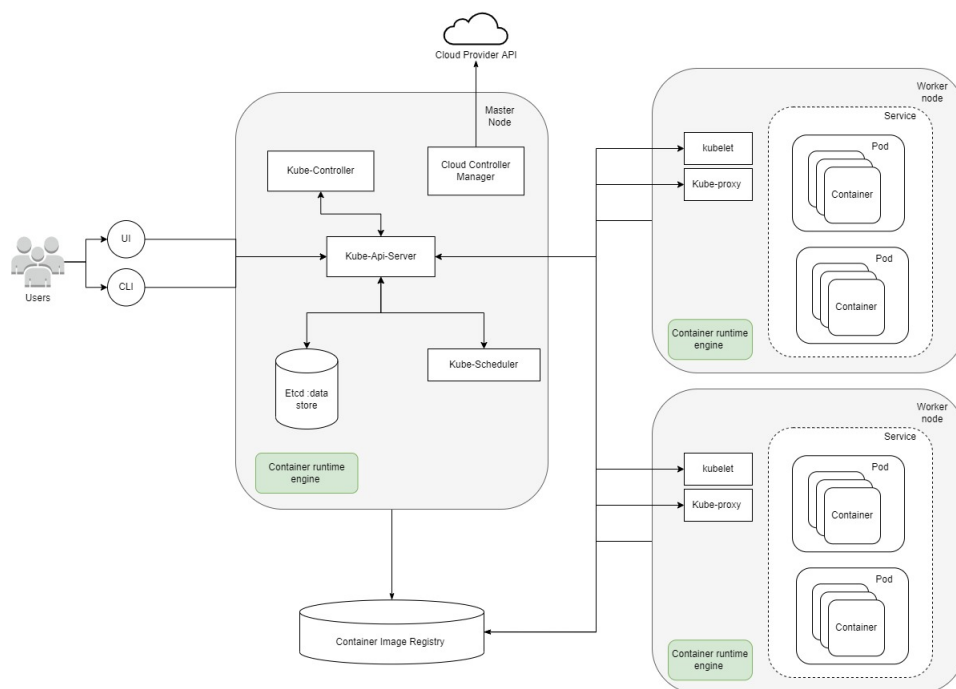


Figure 2.2.1: Kubernetes architecture

Kubernetes has become one of the most widely known and used orchestration tools across different cloud platforms. Istio and AWS App Mesh

support Kubernetes as the underlying architecture.

2.3 Service Mesh emergence

2.3.1 Challenges in Microservices

Microservices are modern technology and have limited research available. Especially, very few articles and research papers have been written on empirical research on the topic [30]. Most of them have been written since Docker was released in 2014. Therefore, it is important to understand the limitations and problems with the technology before adopting it. As shown in [37], cloud practitioners describe a variety of challenges in microservice architectures. Some of them are networking between containers, troubleshooting and tracing an issue, maintaining numerous repositories, or finding a logical boundary between microservices [37]. Each microservice has its data, and maintaining consistency across all data sources is a challenge [30]. As the field is relatively new, it is hard to build expertise in the area. Not all organizations are ready to adopt microservice architecture. Many traditional organizations find it hard to adopt microservices and DevOps technologies, as they do not focus on the readiness of the organization for the move [30]. Design, security, communication, increased operational complexity, and testing are some commonly known challenges of microservice architecture [38].

In modern applications, there are thousands of microservices running with multiple replicas of each microservice in a constantly changing network topology [20]. This dynamic nature of the infrastructure can cause concerns, such as packet loss, increased response time, and faulty responses. In this case, it becomes difficult to trace the root cause of an issue, making debugging process hard without proper traceability [39]. A simple API can involve several calls across microservices, thus increasing the number of network calls. As one involves more network dependencies in the application, the idealistic assumptions about the underlying network such as being reliable, secure, and with a non-changing topology or without latency start becoming invalid [24, 40, 41, 42]. Specially, if a malicious entity compromised a single microservice, each API call has to be secure and should not let the malicious entity get control over the entire application [43]. To address the security concerns, authorization policies of the application need to be granular and implemented per microservice [43]. Since microservices can come and

go in the network, a robust service discovery mechanism is necessary to keep the cluster state updated at all times. It is important to ensure the availability of APIs exposed to the consumers [39]. To ensure fault tolerance and reliability, developers need to implement retry and timeout mechanisms as well as rate-limiting across the microservices. This divides their focus between functionality and operational logic. Microservices need error handling, fault injection, and resilience tests to confirm application resilience. As the microservice behaviour depends on inter-service communication, traffic control between microservice becomes an essential need. It is critical to identify the boundary between the microservice capabilities and its runtime requirements [44]. It gives a clear understanding of what features should be centrally applied to all microservices and avoids the repetition of code.

Overall, observability, traceability, reliability, security, service discovery, fault tolerance, resilience, traffic control, traffic shaping, and the unclear separation between functional and operational requirements are the complexities identified in the microservice architecture.

2.3.2 Service Mesh

All the identified shortcomings of microservices architecture caused the emergence of a new technology called service mesh. Arguably, William Morgan first defined service mesh in 2017 as follows: “A service mesh is a dedicated infrastructure layer for handling service-to-service communication. It’s responsible for the reliable delivery of requests through the complex topology of services that comprise a modern, cloud-native application. In practice, the service mesh’s implementation is an array of lightweight network proxies deployed alongside microservices, without the applications needing to be aware.”[45]

The purpose of a service mesh is to decentralize the networking infrastructure between microservices in your microservice [46, 2]. This decoupling recognizes the boundary between functionality and operational logic of the microservice. With the emergence of the service mesh, developers can solely focus on writing the code for the functional requirements of a microservice. They do not have to implement operational logic such as routing control, retry and timeout mechanisms, or handling authentication and authorization. Service mesh adds resilience, security, and observability features to the application stack [45].

Service mesh is a communication layer that sits on top of microservices. This layer can be implemented in three different ways, namely, as libraries,

node agents, or sidecars, as shown in Figure 2.3.1. In the library approach, each microservice maintains a copy of the library designed to provide all functionalities of the service mesh. This was the most straightforward and original approach taken by companies pioneering cloud-native architecture adoption. Twitter's Finagle, Netflix's Hystrix, and Google's Stubby are some examples of the initial implementation of the library approach [47, 48, 45]. The advantage is that the resource allocation is handled by the OS as the library runs as code inside the microservice. The trust boundary is small, as the microservice trusts the code running in it and not some other entity in the network [49]. In the case of polyglot applications, this approach involves more work, as multilanguage libraries require more work to replicate functionalities in different languages. Also this approach is not compatible if third-party applications are used, as injection of libraries is a challenge in that case [50]. It is hard to maintain libraries as version control issues may surface in multi-cluster implementations [3]. It also causes some performance issues as the library code is embedded in the microservice.

Node agents are an alternative approach that is easier to maintain. It requires an agent running on each node in the cluster as a daemon. This approach is language-independent and serves multiple microservices running on a single node, but requires infrastructure-wide coordination to deploy the agent [49]. It is cheaper to distribute one copy to each node rather than to every single pod, which makes it a performance-friendly approach [3]. It uses the work resource-sharing principle where it allocates resources to microservices on a request basis. Although this provides more efficiency than the library model, it opens room for resource abuse [3, 49]. A single microservice can hog up as many resources as it requests, starving other microservices. Developers have to maintain the logic on the application level to handle resource sharing efficiently. It also creates a single point of failure for all pods running on a single node [51]. If the node agent is lost, the communication to all pods on the node is lost. Linkerd deployments, Kubelet in Kubernetes, and CNI daemon-like flannel follow the node agent architecture [51].

The third approach is the sidecar pattern that launches the communication layer as a container for every container in the cluster, calling it a sidecar. It provides trade-offs between library and node agent approaches. From a security perspective, it follows the least privilege principle, meaning each sidecar is responsible for authenticating requests coming to a single container associated. Whereas, node agents manage security for all containers running on the node, increasing the trusted scope and thereby chances of intrusion by impersonation attacks. Sidecars usually run on the loopback interface,

sharing the network IP and port space with other containers in the same pod. Therefore, other pods cannot directly communicate with it. Configuration to enforce traffic control and routing policies needs to be distributed across all sidecars, which can be an overhead if the scope of configuration is not well-defined. Nonetheless, this model provides a balance between features, performance, and security. This model is implemented by Istio envoy proxies, discussed in detail in the upcoming Section 2.5.1.

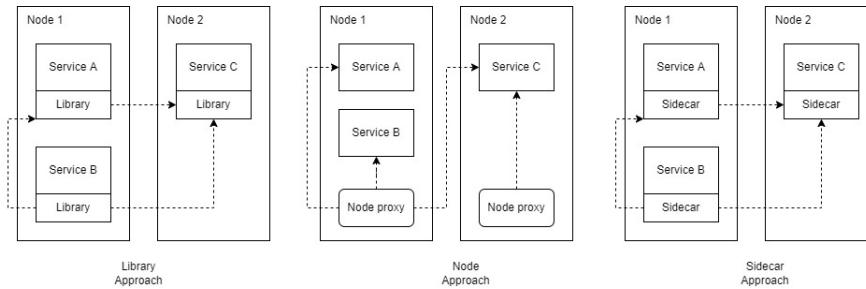


Figure 2.3.1: Different Service mesh implementations [3]

The sidecar proxy pattern is a commonly used service mesh, architecture model [44]. A generic service mesh architecture, as shown in Figure 2.3.2, has a control plane and a data plane. The control plane is responsible for enforcing network policies and distributing configurations. It is also responsible for encrypting inter-service communication and handling routing between the services. The Data plane consists of all the sidecars deployed per pod. It does the heavy-lifting work of intercepting incoming and outgoing traffic to and from the pod. It is also responsible for service discovery and registration, monitoring, implementing authentication and authorization, performing health checks, and load-balancing. To manage the communication, the control plane communicates with the data plane to propagate, translate different policies, forward, and analyze network traffic to/from pods.

A service mesh should be able to handle the following features without having to modify any functional logic of the underlying microservices [2]. Commonly known features of a service mesh are as follows [2, 39, 4].

1. Observability

- **Feature set -**

- Metrics collection - Request latency, bandwidth usage, requests rate

- Graphical visualization - Application health, performance, request

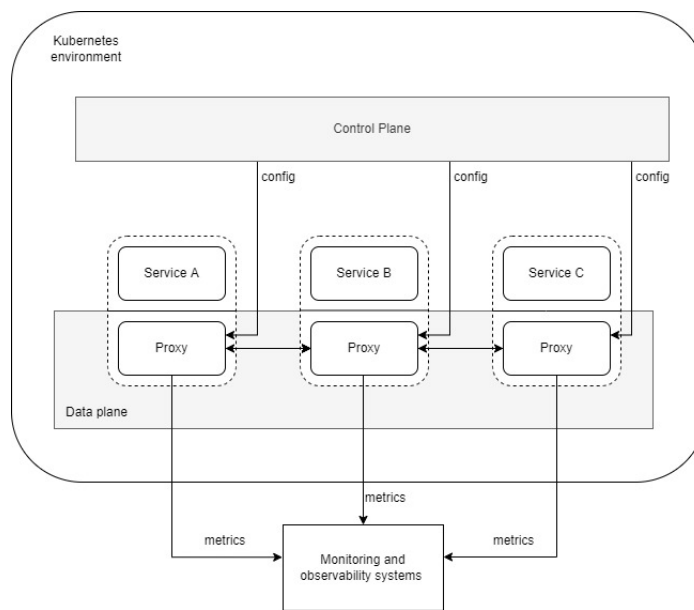


Figure 2.3.2: Service Mesh Architecture [4, 2]

flow, request volume per target

Fault identification - Logging and tracing

- **Description -**

For debugging purposes, it is essential to intercept incoming and outgoing traffic of the pod. This information is useful to visualize microservices statuses, such as if a service is overloaded, or to understand success and failure rates and factors influencing these values.

2. Routing

- **Feature set -**

Traffic management - Traffic splitting, traffic shaping, traffic interception, traffic mirroring, rate-limiting

Testing - Fault and error injection, A/B testing

- **Description -**

Canary and Blue-green deployments, alpha, and beta testing are some of the most time-consuming activities in the Software development lifecycle. With the help of a service mesh, traffic can be easily configured to conduct different scenarios designed for testing.

3. Automatic scaling

- **Feature set** -
Scaling - Scale up or down pods based on error rates and request latency
- **Description** -
The microservice workloads, that is, pods in the service mesh automatically scale depending on the incoming workload.

4. Separation of duties

- **Feature set** -
Development role - Focus on business logic
Operation role - Focus on inter-service communication and managing network and security policies.
- **Description** -
Service mesh enables developers to concentrate on the functionality of the microservice. It should not be the developer's responsibility to configure an inter-service communication process.

5. Trust

- **Feature set** -
Authentication and authorization, Encryption for inter-service communication
- **Description** -
To set the least privilege access model, rules are enforced to allow access between microservices, that is, not every service should be accessible from every other service. Service mesh offers a way to authenticate the end-user and encrypts the inter-service traffic. (example mTLS)

6. Automatic service discovery and registration

- **Feature set** -
Use of underlying orchestration tool (example – Kubernetes), Use of an external service registration tool.
- **Description** -
A mechanism is needed for consumers to discover dynamically

changing locations of microservices. A service registry in an orchestration tool keeps track of newly added and removed instances of such ephemeral services.

7. Resilience

- **Feature set** -
Load-balancing, Circuit breaking, Implement retry and timeout logic
- **Description** -
Service mesh handles failures to ensure resilience and maintains reasonable QoS for the end-users. Modern service mesh load balancing algorithms consider several parameters such as health status, request latency, and current load variable of the node to forward requests to the most ideal node. A service mesh provides load-balancing capabilities at the application layer (layer 7) unlike the underlying tool that provides load-balancing at the TCP/IP layer(layer 3).

2.4 Selection of service mesh implementations

2.4.1 Meshes

As per InfoQ's studies, in 2013, Airbnb released SmartStack, which utilized the service discovery mechanism offered by HAProxy. From there onwards, many companies started exploring the idea that a separate communication layer can exist to manage network-related operations for microservices. In the 2000s, Google developed stubby, Google Front End, Global software load balancer, and gRPC, the concepts that have been referred to in Istio. In the 2010s, Twitter started working with scala-powered Finagle, which later turned into Linkerd. Netflix explored the sidecar pattern in a proxy called Prana. In 2016, NGINX came up with the fabric model that required NGINX products for implementation, but the concept was similar to service mesh [52].

Existing books, papers, and online resources point out Istio, Linkerd, and Consul as the most popular and widely used open-source implementations of service mesh. There are other meshes such as Kuma, and Traefik

mesh(formerly known as “maesh”), which are less commonly used implementations. AWS App Mesh is another example that is offered as a service in the AWS cloud provider. There are other implementations, such as the network service mesh built for telcos and ISPs, OpenShift service mesh by Redhat, Open service mesh, NGINX service mesh, and Apache service comb, which provide a similar feature set with minor differences [53, 54].

2.4.2 Selection Criteria

As discussed earlier, there are several options available to implement the service mesh, but there are certain factors that need to be well-thought-out when considering using a service mesh. It is not a straightforward decision, and it has long-term effects on the development process and software life-cycle. In most cases, a solution with a rich feature set seems a favourable option, but it could also bring technical complexities to the applications [53, 54]. In some use-cases, implementing a library or adopting the architecture could be a good alternative to introducing service mesh, which is harder to learn and implement. It is also essential to compare the performances of different meshes to decide which one suits the best for a specific use case. The choice of a service mesh is highly dependent on the requirements of the applications [54]. After prioritizing these requirements, it is easier to select two or three possible implementations and conduct performance testing on them to compare further outcomes.

Based on the points discussed in the [53, 54], we devised a set of questions and answered them after discussing them with a Development team at Infor, as described in Appendix A, to identify possible options for service mesh implementation. Table 2.4.1 shows the list of general features and whether they are supported by service mesh implementation candidates. From this data, Istio, Linkerd and AWS App Mesh come across as the most suited choices. Istio has all the required features and also supports additional features such as pre-built integrations with Prometheus and Grafana dashboard for observability. It is an open-source service mesh and can be implemented on a variety of platforms, including AWS and Azure. It has a rich set of features that go beyond the required feature set.

Currently, In AWS App Mesh, service-to-service authorization rules are implemented using IAM roles for user authorization. For fault injection, AWS has a fault injection simulator which is a fully managed AWS service that helps simulate faults to verify application resilience, security, and performance [55]. On the other hand, it does not have support for other cloud providers,

and it is tightly coupled with other AWS services. Infor base services are built on top of AWS services, which enable automated configuration of different AWS services necessary to create infrastructure and administrative roles for customers [56]. Therefore, AWS App Mesh is a valid option for implementation.

Linkerd has a solid feature set, and it is quite popular in the developers' community as it is considered easy to install, is less complex, and it provides limited features. However, the missing features could become quite important features on the roadmap of the application. Ingress and egress traffic management is easier in Istio as compared to Linkerd [57]. Other meshes were not considered as they lack a feature set. Although App Mesh does not have all the features supported by Istio, it offers some alternatives, and it is well integrated with other services in AWS which makes it easy for Infor products as they are utilizing the AWS cloud provider for various use cases. Therefore, Istio and App Mesh are the selected service meshes for the thesis implementation.

2.5 Istio

IBM, Google, and Lyft have announced the launch of Istio [58]. Istio defines a standard approach for managing microservices' traffic flow, enforcing access policies, and aggregating telemetry data [59].

2.5.1 Architecture

On a high level, Istio consists of a data plane and a control plane ,as depicted in Figure 2.5.1, similar to service mesh's generic architecture. Istio has a third plane called the management plane which is dedicated to governance, financial management, metering, and multi-mesh management services [5]. The multi-mesh manager is an example of the management plane [60]. Supergloo is another example that provides a management plane to discover multiple clusters and manage control planes [61]. It queries different clusters to see workloads and services on different clusters. The Data plane consists of envoy proxies inserted into each pod, and they intercept all the traffic towards or from the pod. Application containers in the pod are completely unaware of the data plane proxy's existence. The Data plane is responsible for load-balancing, health-checking, authentication, authorization, and collecting a variety of metrics from the application pod [5, 6, 41]. The control plane is the

Table 2.4.1: Support for features per service mesh implementation

Quality	Feature	Istio	AWS App Mesh	Linkerd
Observability	Access log collection	✓	✓	□
	Metrics generation	✓	✓	✓
	Dashboard	✓	✓	✓
	Preconfigured tracing backends	✓	✓	✓
Trust	mTLS	✓	✓	✓
	Service-to-service authorization rules	✓	□	✓
Routing	Load-balancing	✓	✓	✓
	Percentage based traffic splitting	✓	✓	✓
	Header and path-based traffic splits	✓	✓	□
Resilience	Circuit breaking	✓	✓	□
	Retry and timeout	✓	✓	✓
	Fault injection	✓	□	✓
	Delay injection	✓	□	□
Platform	Support for Kubernetes	✓	✓	✓
	Cloud integration with AWS	✓	✓	✓
	Cloud integration with Azure	✓	□	✓
	Mesh expansion	✓	✓	□
	Multi-cluster support	✓	✓	✓

administrative plane that is responsible for data plane proxy configurations. It provides policies and configurations to a set of envoy proxies to create a mesh out of a set of isolated and independent proxies [5, 6, 41]. The control plane provides API or CLI access to provide configuration changes, possibly through a version-controlled CI/CD pipeline. It is the brain of the mesh system that does not touch any network packets, but implements logical boundaries and enforces routing or resilience behaviour through data plane proxies [5, 41, 62].

Istio control plane components consist of Pilot, Galley, Mixer, and Citadel. The **Pilot** is the head of the control plane that keeps synchronization with the underlying container orchestration platform, Kubernetes in a majority of the cases. It translates the information received from other components into xDS objects that are understandable by the proxies. It maintains the locations of all the pods by interfacing with the Kubernetes service discovery mechanism [5]. Furthermore, it is also responsible for traffic management and resilience capacities, such as circuit-breaker and retries logic implementation

[62]. **Galley** validates, manages, and distributes Istio configurations to other control plane components. It uses a mesh configuration protocol to distribute these configurations [5, 62]. It essentially isolates other components of Istio from gathering data from Kubernetes. The **Mixer** is responsible for invoking applied routing and traffic policies. When a service call is made between services or from outside the cluster, the mixer is invoked to verify the authorization policies and quota of requests to allow access. It manages telemetry reporting after the completion of a request [5, 41] and it enables Istio to work with multiple backends and environments by providing abstraction between infrastructure backends and other Istio components [5, 62]. **Citadel** manages the security between services by implementing mTLS to encrypt inter-service traffic. It is responsible for end-user and inter-service authentication. Citadel agents send certificate-signing requests, which are then signed and approved by Citadel's CA component. It is responsible for generating, deploying, rotating, and revoking keys and certificates. It can integrate with an organization's PKI infrastructure to perform key validation operations [5]. Istio data plane proxy, which is the envoy proxy, is introduced in each pod as a sidecar that is configured to observe all the incoming and outgoing traffic to and from the pod. With the intercepted traffic information, the proxy can evaluate access policies, gather metrics, encrypt data, perform health checks, circuit-breaking, terminate TLS, gRPC, and apply traffic control.

2.5.2 Core functionalities

Core functionalities of Istio include the summary of blog [63]:

- **Container orchestration:** Istio orchestrates containerized deployments on the Kubernetes platform and also updates them.
- **Traffic route management:** Istio supports configuring a variety of routing rules to define communication between microservices on a fine-grained level. It supports zone-aware traffic distribution and load balancing along with reliable failovers for HTTP/1.1, HTTP/2, gRPC, and TCP traffic. Istio supports canary and blue-green deployment, as well as A/B testing.
- **Policy management:** Istio provides a pluggable policy layer to enforce security and traffic rules, along with rate limits and available quotas.

Background

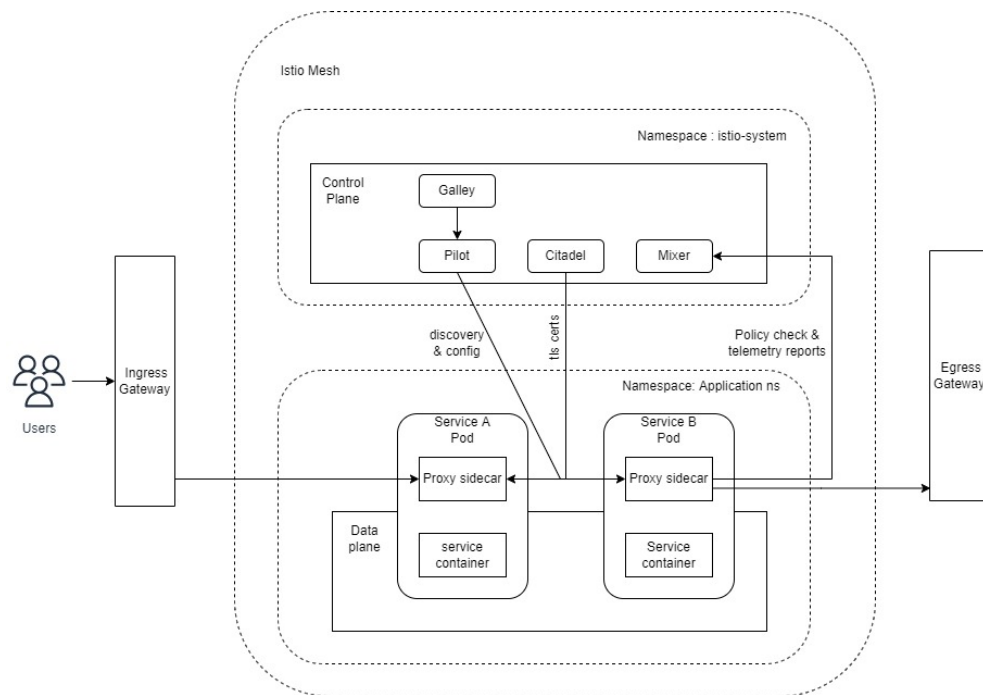


Figure 2.5.1: Istio service mesh architecture [5, 6]

- **Proxy services:** Istio provides proxies for both server and client-side. The client-side proxies manage service discovery, credentials injection, load balancing, monitoring, and logging. The server-side proxy manages rate-limiting, flow control, protocol translation, authentication and authorization, and monitoring and logging.
- **Security:** Istio enforces security measures by providing mutual TLS authentication between the services, which entails them a strong sense of identity.
- **Observability:** From all the metrics, logs, and traces obtained from the traffic, network policy enforcement decisions can be strengthened. Istio provides out-of-box monitoring and tracing solutions such as integration with Prometheus, Grafana, Kiali, and Jaeger.

Istio introduces several custom resource definitions (**CRD**) with its installation. These CRDs are defined in manifest files written in YAML syntax. With the help of `kubectl` commands, these CRDs are then applied along with the application. The CRDs relevant to this project are -

- **Virtual Service** – With the help of destination rules, it provides routing capabilities to the service-specific pod. Virtual service configuration is essential for A/B testing where traffic is forwarded to different versions of application service from a single virtual service. Based on the order of the destination rules, Istio evaluates them to forward the incoming request to the correct target application service [64].
- **Destination Rules** - They define subsets to group different versions of a single application service. Based on the specified weight to each subset, A/B testing or canary deployments are implemented to switch between the target versions. Destination rules also allow defining traffic policies such as load balancing, TLS configuration, retry, timeout, or circuit breaker settings. It also allows the configuration of fault injection to introduce delays in response to test application resilience [64].
- **Gateways** - Ingress and Egress gateways are used to handle traffic at the edge of the service mesh. Ingress gateway allows traffic management configuration such as TLS configuration and ports to expose outside the mesh. The egress gateway can be configured to limit the services which can have access to the external network [64].

2.6 AWS App Mesh

AWS announced a new service called AWS App Mesh in November 2018, which later went to General Availability(GA) in March 2019. It is not an open-source service, but it uses Envoy proxies as sidecars, the same as Istio, which are open source and free of cost [2].

2.6.1 Architecture

AWS App Mesh architecture follows the sidecar pattern similar to Istio. On a high-level design, it contains a control plane and a data plane. AWS App Mesh service is a control plane hosted to manage and configure Envoy data plane proxies. AWS App Mesh consists of the following components [65, 66].

1. **Service Mesh:** It is a logical boundary encompassing all the network traffic flowing between services that reside in the mesh. It sets a logical component to identify different entities such as virtual nodes, virtual services, virtual routers, and routes as well as data plane proxies.

2. **Virtual Node:** It is a logical pointer towards the actual discoverable service running in EKS or EC2 instances or ECS or AWS Fargate [8]. Creating a virtual node is not enough in itself to enforce different policies and configurations. It is a component that exposes the underlying service to the service mesh architecture. Every service in the mesh has a virtual node pointing to it.
3. **Virtual Service:** It provides an abstraction of the discoverable service using a virtual node or virtual router with different routes enabling traffic forwarding to the discoverable service. The names of the virtual services are discovered using the actual name of the discoverable service. This way, services discoverable before the implementation of service mesh do not change the logical names.
4. **Virtual routers and virtual routes:** They handle traffic for one or more virtual services defined in the mesh. Each router has one or several virtual routes associated with it. A route matches certain rules defined on the router to forward the request to the associated virtual node. The traffic routing criteria include URL paths, HTTP headers, gRPC service, or method names. Traffic can be routed to one virtual node and changed to another over time, or a percentage of traffic can be defined for each virtual node. This enables canary, blue-green deployment as well as A/B testing.
5. **AWS App Mesh Controller:** It keeps a watch on changes in the custom resource definitions of mesh, the virtual node, virtual service, virtual router, and routes for creation, modification, and removal. It makes the changes in the corresponding CRDs through App Mesh API. The controller injects, the Envoy proxy and proxy route manager into the pod [67, 68].
6. **Envoy proxies:** They are deployed using Envoy data plane sidecar proxies along with each pod. For changes in configurations to the service mesh, there is no need to change or redeploy services. They apply received configurations to direct traffic appropriately to defined targets. All traffic between the services is intercepted at this component.
7. **Proxy route manager:** It runs as an init container in the application service pod and updates the IP table rules in the namespace to route the traffic through the envoy proxy in the pod [67].

Background

The components of AWS App Mesh are installed using CRDs provided for Mesh, VirtualNode, VirtualService, VirtualRouter, Virtual gateways, and GatewayRoute, as depicted in Figure 2.6.1 [68, 67].

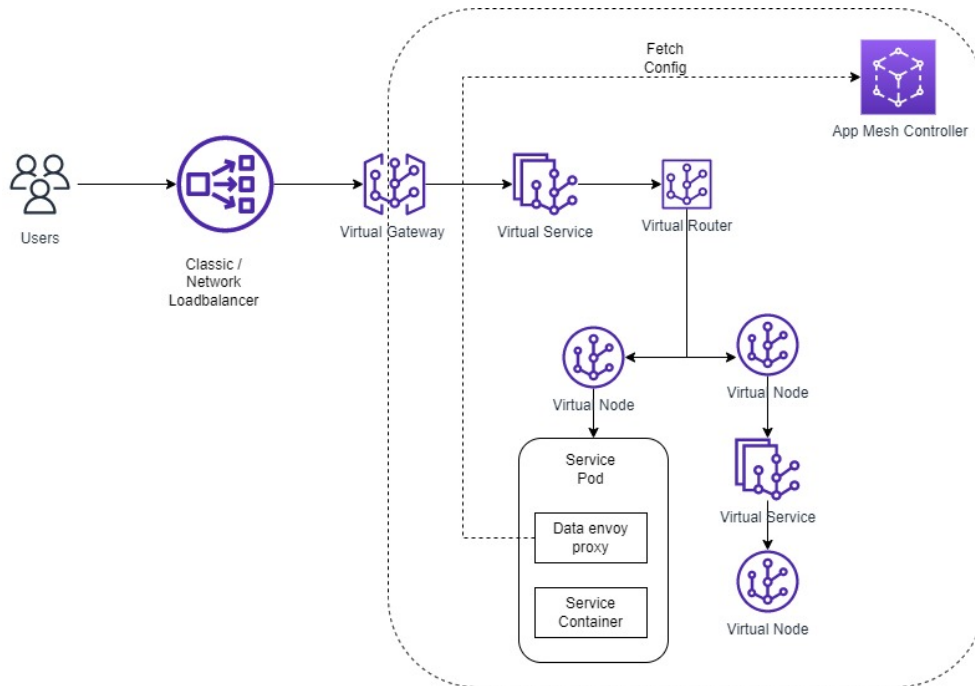


Figure 2.6.1: AWS App mesh components(CRDs) [7, 8]

2.6.2 Available and selected features

- **Container orchestration:** App Mesh is compatible with AWS ECS, AWS EKS, and AWS Fargate running application workloads. Every service starts an envoy proxy sidecar in the pod to register with the service mesh [69, 70].
- **Traffic routing:** App Mesh allows configuring traffic rules between services. With this feature, traffic can be moved to a new version gradually or completely by specifying the percentage of traffic towards each version of the same service. Canary, blue-green deployments and A/B testing are possible because of this feature. Connection timeout and automatic retries can be configured at the proxy level [69, 70].

- **Client-side traffic policies:** App Mesh Envoy proxies load balance requests incoming from all the clients in the mesh. It manages adding healthy targets or removing unhealthy targets from the service backend to ensure resilience to request failures [69, 70].
- **Fully managed:** App Mesh is a fully managed service that inserts Envoy proxy in each service pod. No extra infrastructure installation is required to manage communication [69, 70].
- **Observability:** App Mesh collects application metrics such as access logs, traces, total requests, Queries per second (QPS), and response latency. Metrics and traces can be analysed further by the AWS X-ray service to provide more insights into the application performance. It is also compatible with open source monitoring and visualization tools such as Prometheus and Grafana, as Envoy proxy is open source [69, 70].
- **Encryption:** App Mesh supports inter-service traffic encryption with a client-provided TLS certificate or AWS certificate manager to keep the communication secure. With mutual TLS, it authenticates one service with another [71]. It handles TLS termination, negotiation, and rotation and keeps application logic free from these activities.

2.7 Related work

In the field of service mesh technologies, not a lot of research has been conducted to study the technology and its application impacts thoroughly. There are plenty of blog posts, online content, websites, and YouTube videos explaining the working of the service mesh, but the research in the area regarding performance implications and resource utilization of service mesh architecture is lacking. Sources related to competitive analysis of service meshes were studied to design the experiments conducted during the thesis implementation [72, 13, 11, 73, 74, 75]. Most of the found sources are not scientific research papers, therefore they lack certain details and their credibility could be questioned. The reason to refer to these sources is to understand the basis of several experiments conducted in the competitive studies of service mesh. With this information, a strategy to perform experiments for the competitive study of Istio and AWS App Mesh is designed.

In this section, RPS is used interchangeably with QPS, as essentially both mean the same for a generic request-response system [76].

2.7.1 Performance benchmark and resource utilization

Istio and Linkerd are the most popular implementations of service meshes. Major sources concerning performance evaluation and comparison involve these implementations [72, 13, 11]. AWS App Mesh and Istio comparisons mostly involve theoretical work with feature comparison [73, 74, 75, 10]. Specially, three main sources were found that compare Istio and Linkerd in terms of performance and resource utilization. The idea is to study the strategies used by these sources and identify important takeaways that can be helpful to design a comparison strategy for Istio and AWS App mesh.

The first source is the master's thesis written by Erik Dahlberg, summarized by Lars Larsson, which compares Istio 1.5.0 and Linkerd 2.7.1 in terms of features, response latency for the increasing number of users with constant throughput, and CPU/Memory utilization of overall cluster resources [11]. The benchmarking setup involves 4 virtual machines of type n1 standard 4 that have 4vCPU and 15 GB RAM on the GCP cloud platform. Istio is installed with a demo profile that is designed to showcase Istio's functionalities [77]. A test application called TeaStore with 7 microservices is deployed on the GKE cluster.

Apache JMeter is used as a load generator tool that produce a constant request throughput, which is the number of requests generated per second using 8 different HTTP requests. Latency is measured through Jmeter, and CPU/Memory resource utilization is measured with node exporter and Prometheus. The benchmark is run 3 times to ensure confidence in the results. Every benchmark has 5 stages where the number of users is increased by keeping a throughput of 5 requests per second (RPS). Initially, 20 users are launched to warm up the setup for 3 minutes and then each user group performs load testing for 14 minutes, with 5 minutes of cooling time between two stages. There is no explanation found for the numbers chosen for the benchmarks. These benchmarks are performed on applications deployed without any mesh, with Istio mesh, and with Linkerd respectively. Based on the graphs, approximate data is mentioned in Table 2.7.1 -

As compared to the setup without any service mesh, Istio causes 133% more latency, 131% more CPU utilization and 45% increase in memory utilization, whereas Linkerd causes 175% increase in latency, 97% more CPU utilization and 11% memory utilization during the 20 users stage run. The report concludes that Istio is a better option when it comes to maintaining

Table 2.7.1: Statistics collected from graph approximations found in [11]: Latency (P99) in milliseconds, CPU in millicores, and Memory in GB

	Bare			Istio			Linkerd		
Users	Latency	CPU	Mem	Latency	CPU	Mem	Latency	CPU	Mem
5	30	1000	9.20	51	2000	12.10	75	1900	10.10
10	32	1500	9.70	60	3000	13.00	80	2800	10.60
20	35	2200	9.80	90	5200	14.20	100	4300	11.00
30	37	3000	9.90	100	6800	14.80	160	5800	11.00
40	51	3800	9.95	135	8300	15.00	351	5800	11.10

certain throughput with low latency at an expense of higher CPU utilization. Linkerd, on the other hand, is more suitable when the setup has lower resources. It is not clearly stated which functionalities in Istio/Linkerd are implemented with the application. With TLS and other functionalities in place, the CPU/memory usage might be different depending on the intrinsic behaviour of the implemented functionalities. Another observation is that the overall CPU/memory is calculated for the cluster instead of considering CPU/memory usage per control plane and data plane to have more visibility in the cluster. Because of that, it is difficult to understand which component is causing more consumption, resulting in a CPU/memory bottleneck. It is mentioned that a demo profile of Istio is used, but it is not designed for performance testing. Furthermore, it is also not advised to be used in production, and it is not mentioned whether all components of the control plane are enabled, which could be the cause of higher resource consumption.

The second source is the blog post on performance analysis comparison between Istio 1.1.6 and Linkerd2-edge-19.5.2, written by Thilo Fromm [13]. This report concerns a comparison of the performance of Istio with all control plane components(stock), Istio with several components disabled in the control plane(tuned), and Linkerd focusing. It mainly compares the resource cost and performance impact of service mesh architecture. Buoyant, the company that created Linkerd, approached Kinvolk to compare Linkerd with Istio. It is important to note that Kinvolk had an ongoing client work with Istio in 2019, hence the motivation behind this comparison is mainly to improve open source technologies available for service mesh implementation. The goals of this report include defining an open-source benchmarking framework to measure operational costs per implemented service mesh architecture for the designed scenarios. The strategy of testing is to load-test the application to

a point where response latency is reasonable, and user experience does not suffer. Despite the overall system being subjected to load-tests, the application, and the mesh, an attempt is made to make sure that the response latencies are pushed to the acceptable range and not beyond. RPS are kept constant and response latency gauges the performance of the service mesh. There are two limitations in this scenario, one is that the requested load (RPS) will never be generated. If there is a delay in sending one request, it will eventually slow down the consecutive requests. This triggers a domino effect that slows down consecutive processes of sending requests, waiting for a response and sending another request. Secondly, during this scenario, the service mesh gets some idle time from processing, as there could be a point when it has no request received as the response to the previous request is in flight. To avoid this problem, the author forked a project `wrk` and created `wrk2`, which makes an effort to compensate for the lost RPS by temporarily issuing double the expected request count. P99 latency is important as it directly affects the user experience. If one request gets delayed, the operations in the single request that might be fetching multiple data also get delayed. P99 for individual requests becomes more common for more complex operations, making it an essential metric. Resource consumption is another significant metric, as service mesh provides a wide range of features by consuming CPU/memory that ideally is used for business logic operations. Lokomotive is a tool used for infrastructure provisioning and automating performance testing [78].

The benchmark setup disables some components, such as mixer's policy feature, tracing, gateways, and Prometheus to compare Istio with bare minimum components to reduce resource utilization. This tuned Istio is not considered for the thesis implementation as the production setup will have a lot more functionalities and observability tools which require all components from the control plane. The benchmark setup involves 5 worker nodes of type `c2.medium` with 24 vCPU and 64 GB RAM on Packet IaaS provider. Performance testing and latency measurement are conducted from within the cluster to avoid latencies introduced by ingress gateways. A sample application provided by Linkerd called `Emojivoto` is deployed. The initial replicas of each pod are not specified in the report. Multiple clusters across different data centres are used to eliminate volatile network effects and achieve consistent results. After deploying the mesh and load generator `wrk2`, tests are performed and metrics are collected every 30 seconds until the tests finish. Then the mesh and the generator are removed, and another mesh is installed to perform the same steps again. The experiment is conducted multiple times to gain statistical coverage. Ten minutes long benchmark with varying RPS on 30

application replicas (90 microservices in total) is run to identify the point where some failures are occurring. Some failures start to occur between 500 RPS and 600 RPS. The setup is subjected to 500 RPS for 30 minutes to get the limit of the mesh where performance is acceptable at the highest point of load it can sustain. After that, 600RPS are generated for 30 minutes where the maximum limit of acceptable latencies are observed for Linkerd whereas for Istio the measured latency is in minutes. Table 2.7.2 summarizes the results for a 500 RPS test conducted for 30 minutes.

Table 2.7.2: Logarithmic latency comparison

Latency percentile	Log latency in milliseconds		
	Bare	Istio	Linkerd
p50	2	4	5
p75	3	5	6
p90	4	6	7
p99	5	800	8
p99.9	6	1050	900
p99.99	600	4000	1000

Istio has higher resources than Linkerd, but Linkerd uses fewer resources by handling 600 RPS without needing to autoscale. Istio isn't able to provide 600 RPS, but can manage up to 568 RPS. Overall, istio performs well under high load, keeping the latency to an acceptable amount.

The third source is the performance benchmark written by Ivan Sim that compares Istio 1.0.3 and Linkerd2-edge-18.11.1 in terms of response latency and request throughput [79]. Fortio 1.3.1 is chosen as a load generator, echo server, and report generator tool due to minimal dependencies. It supports HTTP, HTTPS, gRPC, TLS protocols in benchmarking tests. The benchmark setup contains 5 nodes, where 4 nodes are of type n1-standard-1 and the remaining of type n1-standard-2. One node with a higher configuration is dedicated for control planes of both Istio and Linkerd. Two Fortio echo servers are deployed on 3 machines, one for a baseline that is without any mesh and the other two for Istio and Linkerd. The load generator is launched on a dedicated machine. Gke.sh script is used to automate the entire testing strategy including the provision of the machines, testing, collecting metrics, and reporting [79]. It first provisions all 5 machines with respective components and runs a series of tests where HTTP and gRPC load tests are performed on all 6 echo servers, 2 dedicated to baseline setup without any mesh, 2 for Istio-meshed setup, and 2 for Linkerd-meshed setup. Test 1 and test 3 were performed to measure the

Background

maximum HTTP and gRPC request load echo servers could sustain. Test 2 and test 4 were performed using 75% of the lowest QPS value to avoid a warning related to sleep falling behind events when testing against Istio, which gave the lowest QPS in test 1 and test 3. The purpose of the test is to see how the system behaves with not extreme but still considerable load, in order to not break the system but see the performance under manageable load. Sleep falling behind warnings occurs when the required QPS is higher than what your system can sustain.

Table 2.7.3: Test data for different service mesh implementations. The number of connections for each test is 32.

Tests			Baseline			Istio			Linkerd		
protocol	QPS	time	p99.9	p75	QPS	p99.9	p75	QPS	p99	p75	QPS
HTTP	0	30sec	5.6-6.2	1.2	30-35k	32.5-72.6	10.2	3.2-3.9k	8.6 to 11.1	2.7	10-12k
HTTP	2374	5min	5-6	1.1	2374	35-55	10	2374	8-12	2.4	2374
gRPC	0	30sec	9.2 - 12	2.6	14k-16k	30-75	10.4	2.8k-3.6k	11.5-16.5	4.2	6.5k-8.5k
gRPC	2113	5min	9-11	2.3	2113	36-45	10.1	2113	11-14	4.2	2113

For both Istio and Linkerd2, the latency was higher, and the throughput was lower in all tests as compared to the baseline setup. The latency incurred in Istio meshed setup was higher than Linkerd meshed setup in both HTTP and gRPC tests. Similarly, the throughput was lower than Linkerd.

The conclusions of the papers are not of uttermost importance in the thesis context, as they essentially compare Istio with Linkerd and not AWS App Mesh. The results from different sources are subjective to the setup details such as cloud platform, network latencies, number of worker nodes and their configurations, nature of the tests executed and tools used to collect and analyse data. Therefore, it would not be a fair or accurate comparison if the results from these different sources are compared with each other. From these results, a common observation is that Istio control plane components are resource-intensive. In Dahlberg's experiment, the time to the first byte is considered latency, that is, the round-trip time to receive the first byte of the response [80]. P75 latency is then calculated while plotting graphs and making an analysis. The reason for doing so is not mentioned. In Fromm's blog post, we can see that p50, p75 and p90 latencies are smaller for Istio than Linkerd, whereas, P99 and P99.99 latencies drastically increase for Istio. In Sim's blog post, both P75 and P99.9 latencies are larger for Istio than Linkerd. Latencies for baseline are always less than Istio and Linkerd meshed setups in all three sources. P75 is the only comparable latency in these sources as there is not enough data in Dahlberg's master thesis to retrieve P99 or P99.9 values,

although the graph has multiple latency percentiles, there is no description for the same.

Table 2.7.4: Comparison of latency-related statistics, setup details, and test scenario details collected from references

	Dahlberg	Fromm	Sim
Throughput(QPS)	2000	500	2374
Duration(min)	14	30	5
Concurrent connections	40	NA	32
Total requests	28000	15000	11870
Baseline(p75)	25	0.5	1.1
Istio(p75)	70	0.7	10
Linkerd(p75)	160	0.8	2.4
Cloud provider	GKE	Packet	GKE
Load generator location	Within the cluster	Within the cluster	Within the cluster
number of nodes	5	5	1
Node configuration	N1-standard-4	c2.medium	n1-standard-1

Fromm's experiment used the highest machine configuration, but performed an experiment with lower throughput than the other two. After converting the log latencies to the actual values (Antilog of latency to base 10) Istio seems to maintain lower latency than Linkerd at least for p75. The same trend can be seen in Dahlberg's report as well. Contrarily, for a much lower throughput in Sim's experiment than Dahlberg's, Istio causes more latency than Linkerd. This could be due to lower node configurations and a lower number of dedicated nodes per application stack (with/without mesh) but this conclusion cannot be concretely confirmed.

Another article uses Locust as a benchmarking tool to measure the maximum throughput, concurrent users, success and error rates, and response latency a sample application can handle on three different solutions, namely, azure app service, content delivery network, and Azure front door [81]. These solutions are not relevant to service mesh, although the metrics collected and the scenario performed in the article are relevant to design scenarios for the thesis implementation. The scenario is to generate requests with 10 concurrent users and increase the number of users by 10 every second until 5000 concurrent users are spawned. The test is executed for 20 minutes. Data collected from all 3 endpoints are plotted using Locust and the comparison between the three solutions is analysed using metrics such as requests handled per second, median response time, peak response time, and error rates per

solution. This article describes a realistic scenario where RPS is not constant and the number of concurrent users using the application changes over time. This scenario would not give comparable values for the service mesh's resource consumption, as the request throughput will be different for each service mesh. However, it is helpful to test the limits of the application in terms of request latency, request throughput, and success/error rate with believable numbers.

In Fromm's experiment with 500 RPS subjected to the setup for 30 minutes, the CPU utilization and memory consumption are calculated per container over small intervals of time. The median value of the highest values is taken into account from three data points(bare, Istio, and Linkerd) during the tests. Fromm's observations about the CPU difference between Istio and Linkerd are supported by Kipper's experiment [12]. In Kipper's experiment, a custom-made Istio resilience Simulator is created for benchmarking and the benchmark setup involves 4 to 6 nodes of type N1-standard-4 on a GCP cloud in a GKE cluster. Datadog is used as a monitoring tool, although in the setup configuration Prometheus installation is allowed. There is no explanation about the duration of the test in Kipper's experiment. From both the experiments, it can be concluded that Linkerd has the edge over Istio concerning CPU utilization. Both experiments measured resource consumption by removing the mixer component of Istio. In Fromm's experiment, almost 1000 m cores were used, whereas in Kipper's 100 m cores of CPU were used less by doing so. Similar conclusions can be made about memory consumption for Istio components being higher than Linkerd's from Dahlberg's data and Fromm's experiment, but from Dahlberg's data, it is difficult to understand which components are most memory intensive.

Table 2.7.5: Comparison of CPU and memory statistics collected from [12, 13]

Reference name	Experiment	Metric	Component	Istio	Linkerd
Fromm	500RPS for 30 minutes	memory(GB)	Application proxy sidecar	4.6	0.4
			Control plan	3.5	0.5
		CPU(m)	Application proxy sidecar	2300	1600
			Control plan	1400	100
Kipper	100 RPS for 6 hours	No memory statistics collected	X	X	X
		CPU(m)	Application proxy sidecar	310	200
			Control plan	750	22

Background

Chapter 3

Method and implementation

Based on the findings of Section 2.7, it can be concluded that AWS App Mesh is not a highly researched topic. To the best of our knowledge, no articles related to performance comparison between Istio and AWS App Mesh were found at the time of writing the thesis. The scenarios described in other thesis implementations, blog posts, and articles about comparing Linkerd and Istio can apply to benchmark Istio and AWS App Mesh in terms of performance. To design these scenarios, it is important to first drill down the technology details, application details, and toolset that can be used for conducting different processes. The following sections discuss the tools, the implementation of the selected technology, and a thorough description of the benchmark. These sections deal with the core implementation of the thesis.

3.1 Selection of tools and technologies

The comparison between Istio and AWS App Mesh is based on factors such as performance, resource utilization, application visibility, and solution cost. As the service mesh is an essential component due to the features it provides, the comparison does not include any baseline setup without a service mesh implementation.

The idea of the first scenario is to find the optimal request throughput that both the Istio-meshed and the AWS App Mesh-meshed sample application can handle without causing any failures and yet maintaining an acceptable response latency. The average response latency is 1280 ms for desktop applications in 2020, although Google suggests achieving response latencies

under 200 ms [81]. For implementation, we consider 300 ms as the acceptable median latency(p50) [82]. We try to maintain 99 percentiles latency (p99) below 1000 ms, considering server-side processing time and inter-service communication delay. Keeping these numbers in mind, the benchmark is performed to find the maximum request throughput that can be achieved by both Istio and AWS App Mesh. Almost no failures are accepted during this test. Once this optimal point is found, in the second scenario, the benchmark is performed to observe response latency distribution for a constant request rate, which is 75% of the lowest of the optimal throughput achieved from the first testing scenario.

In the third scenario, a prolonged test needs to be run to retrieve CPU utilization and Memory consumption statistics for both the meshes, as resource utilization is an important factor. Both the meshes are subjected to high load for several minutes to get consistent results. Data of memory and CPU utilization is collected per control plane and data plane of both the meshes.

Based on all the data retrieved, Istio and AWS App Mesh are compared with one another in terms of resource utilization, throughput, and response latency. Keeping these 3 different scenarios, the following tools mentioned in Table 3.1.1 are chosen for conducting performance benchmarking. Other tools and technologies are available. For example, wrk/wrk2 and Locust for performance testing, Datadog for monitoring, and ECS or Fargate service on AWS for container orchestration. But, Locust does not provide constant throughput, Datadog is not an open-source tool, and ECS or Fargate service would require changes in the current way of deploying microservices at Infor. Therefore, the technology stack mentioned in Table 3.1.1 is the most suitable stack to conduct the designed benchmarking scenarios.

Process	Tools
Implementation	EKS of AWS
Performance testing	Fortio
Monitoring	Prometheus
Traceability	Kiali, Jaeger, AWS X-ray service
Visualization	Grafana, Kiali, AWS X-ray service

Table 3.1.1: Selected tools and technologies

3.2 Application and cluster details

Experiments mentioned in the related work involve comparing performance and resource consumption of service meshes with plane Kubernetes to understand the magnitude of increased latency and resource utilization. In the thesis implementation, this scenario is not considered, as using a service mesh is a requirement, and the selection of the right type of the same is the objective. For the implementation of different scenarios, it is important to select an application with multiple microservices. BookInfo is a sample application provided by Istio that has 4 microservices to demonstrate various features of Istio, as shown in Figure 3.2.1. All microservices are written in different languages such as Java, Python, Node, and Ruby. Infor's applications are currently developed in the microservice architecture using Spring boot, although as the number of microservice grows, the language will not be limited to Spring boot. Therefore, it could be beneficial to test a polyglot application. This application does not have specialities with Istio, and that makes it deployable with both Istio and the service mesh. The 4 microservices in the application include Productpage, Details, Ratings, and Reviews. Productpage serves as a UI of the microservice and internally calls Details to show book information. Details internally call Reviews to display available book reviews. Details also call ankingto fetch rankings of the book. This application launches 3 versions of Reviews microservice to demonstrate the canary deployment feature supported by service mesh. Review v1 does not make any call to the ratings service, whereas Reviews v2 and v3 call ratings to display ratings in black and orange coloured stars, respectively.

With this application, more features supported by Istio can also be implemented easily, as the application is fully compatible and mainly designed to showcase Istio's capabilities. On the other hand, special efforts are required to deploy the same application in AWS App Mesh. In order to add more functionalities, there is a lack of readily available scripts written for the BookInfo application for AWS App Mesh. Due to the time constraint on the implementation of the thesis, Canary deployment is the only feature selected for deployment on the meshes.

To implement the designed scenarios, a single EKS cluster with version 1.18 is deployed on the AWS cloud platform. This can be done in several ways, such as a Terraform or CloudFormation on AWS. At Infor, we have automation suites designed with the help of CloudFormation to deploy EKS clusters with required configurations. The Provision cluster has 3 nodes, where each node

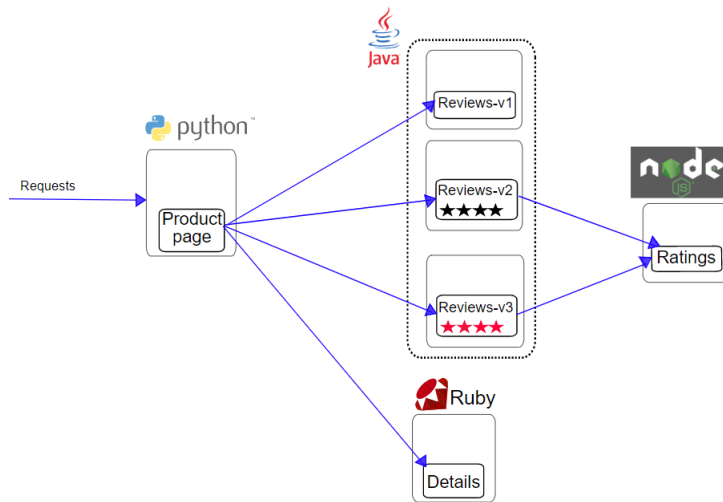


Figure 3.2.1: Application without any mesh [9]

is dedicated to running specific application workloads. Each node is deployed in an EC2 instance and has instance type m5.8xlarge. This instance type gives 32vCPU and 128 GB memory to each node. As discussed in the theoretical background, Kubernetes is an underlying orchestration tool used by the EKS service on AWS. The control plane of Kubernetes is managed by AWS, but the nodes that are deployed are controlled and managed by the user. One node runs the Istio-meshed application, another node runs the AWS App Meshed application, and the third node contains the control plane of both meshes and Fortio pod used for benchmarking. Separate labels are assigned to each node to ensure that the control plane does not hinder the performance of the data plane nodes. Based on these labels, pods of a certain application are assigned to the node they belong to. In this way, the gained data is more consistent and is not polluted due to other mesh or the benchmarking tool. Benchmarking is conducted from within the cluster to reduce the effect of network latency due to the distance between the client and the server.

3.3 Application deployment on Istio and AWS App Mesh

Istio version 1.11.0 is installed with the help of istioctl binary. There are multiple ways to install Istio, such as using helm charts, Istio operator, or

Node	Node Label	Workload	Namespace	Namespace label	Monitoring
Node 1	app=system	Istio control plane	Istio-system	-	Prometheus, Grafana, Kiali, Jaeger
		App Mesh control plane	appmesh-system	-	Prometheus, Grafana
		Fortio pod	fortio	-	-
Node 2	app=Istio	Istio-meshed application	istio-bookinfo	istio-injection=enabled	Istio-proxies to collect data
Node 3	app=appmesh	App Mesh-meshed application	appmesh-bookinfo	appmesh.k8s.aws /sidecar Injector Webhook=enabled, mesh= appmesh-bookinfo	App Mesh data proxies to collect data, AWS X-ray daemons

Table 3.2.1: Cluster and application set up details

direct virtual machine installation. Istioctl binary is used because it was the easiest option with minimal configuration. AWS App Mesh provides a helm chart for easy installation of the control plane. However, it does not provide additional details about the control plane and how it works because it is not an open-source technology. The default profile offered by Istio is used for this experiment because it is production-ready and suitable for performance testing. In both Istio and AWS App Mesh, the control plane can be deployed outside the cluster that runs the application workloads. Istio provides an external control plane installed on a separate cluster that is accessible from the data plane. AWS provides an option to install the control plane as a Fargate service, where the control plane pod runs in a serverless environment [67]. This option was not explored in the thesis, as the purpose of the thesis is to essentially compare the impact of the control planes on CPU and memory utilization. If the control plane is removed from the cluster of workload execution, it might introduce additional latency due to inter-cluster communication overhead between the control and data plane. However, Fargate can significantly impact the overall cost of the solution, depending on the load and the designed scenarios (Discussed in Chapter 5).

The selected feature for implementation is canary deployment. Canary deployment is a way of deploying multiple versions of the applications in the same application stack. This is to ensure that a certain version is thoroughly tested before it is completely rolled out [83]. To achieve this, some percentage of traffic is routed towards a new version and some towards the old. Traffic rate toward the new version is observed, and the user response is tracked to understand whether customers appreciate the new version. Once the customer confidence is established in the new version, more traffic can be sent towards the new version. Eventually, 100% of traffic can be sent to the new version, and the old version of the microservice can be completely removed. The reason to select this feature for the thesis is that it is on the roadmap for Infor's production-grade application. In an actual canary deployment, only two versions of a service are involved, an old and a new version, but in the test scenarios we have deployed 3 versions because there could be 3 release candidates as well. As the feature set grows, the resource consumption of the control plane and the data plane proxies is also affected by the service meshes. Other features such as authentication and authorization, retry and timeout mechanisms, and securing inter-service traffic are also important and need to be implemented in the future, but due to time constraints, only a single feature is selected.

For application deployment on both the meshes, an automation suite is

created, and the code is made public on a GitHub repository [84]. It is used to deploy the entire application stack on Istio and AWS App Mesh, and remove it after testing. The application is deployed with a Kubernetes deployment YAML manifest. This manifest is the same for both the meshes, with some minor changes discussed later. This contains 6 deployment resources to create 10 replicas of each microservice for one mesh. The application has 4 microservices with one of them running 3 different versions of the application, making it a total of 6 microservices. Therefore, altogether 60 replicas are deployed per meshed application. The application base code is not mentioned in the deployment process, as it is already made public by Istio and is also available on the code repository [85]. The deployed application on Istio is shown in Figure 3.3.1, whereas Figure 3.3.3 and Figure 3.3.2 depict the deployed application in AWS App Mesh.

On a plane Kubernetes cluster, canary deployment can be handled by maintaining a specific ratio of the number of replicas of each version of the microservice. In Istio, a CRD called Destinationrule is used along with VirtualService for Reviews microservice, as it has 3 different versions running. All versions are targeted to have an equal load, which can be done in two ways. One way is to use DestinationRule to define a round-robin strategy for load balancing, as shown in Listing 1. Another way is to create a virtual service along with DestinationRule and use subsets to define the weight for each target version, as shown in Listing 2.

For the second approach mentioned in Listing 2, if there are more than two versions, it is more tedious to define exactly equal percentage for all 3 versions. Hence, the first approach is implemented for the thesis, being Productpage the only microservice exposed outside the cluster. Specially, it uses a virtualservice mounted on the default Istio ingressgateway that is installed with the default Istio profile.

The code in Listing 3 makes it possible to reach the application from outside using productpage microservice's user interface. Istio ingress is deployed along with the controller when we install Istio. By mapping a gateway resource onto this ingress, it becomes possible for the user to reach the application.

When deploying the same application on AWS App Mesh, the application namespace is labelled with "appmesh.k8s.aws/sidecarInjectorWebhook=enabled" to allow data proxy sidecar injection in every pod launched in appmesh-bookinfo namespace. A resource of type Mesh is created in Listing 4 to associate the application with the application namespace.

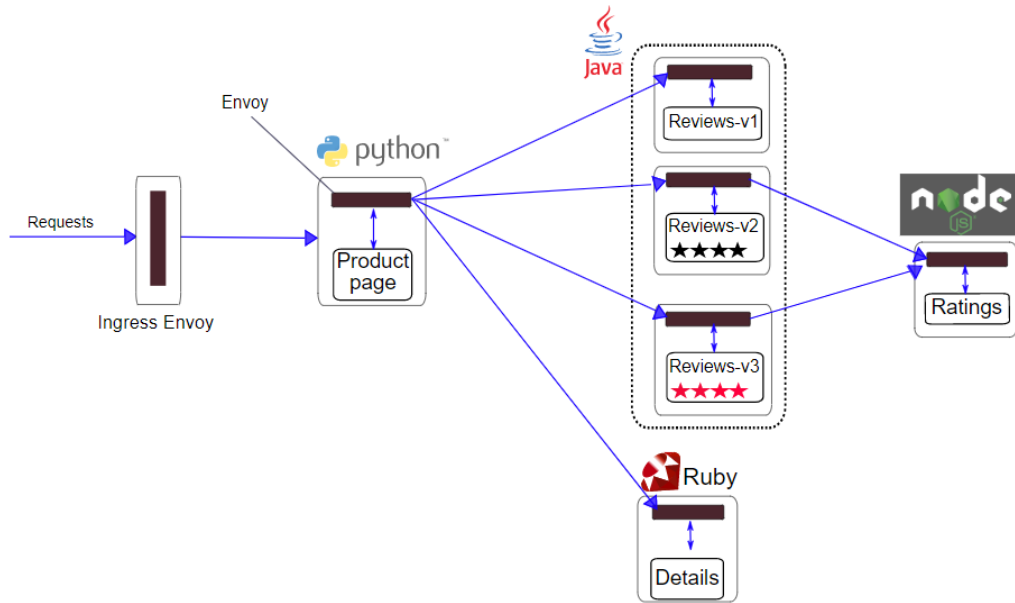


Figure 3.3.1: Application Bookinfo with Istio service mesh

To deploy each microservice in the mesh, three components are deployed, namely, VirtualService, VirtualNode and VirtualRouter with single or multiple routes. In the case of microservice reviews, every version has a separate VirtualNode associated with it. A VirtualService with a VirtualRouter is created to define access policy to these 3 versions. VirtualRouter has routes to 3 VirtualNodes with equal weight in order to allow equal traffic distribution among all 3 versions. Similarly, for the other 3 services, a single route is created, allowing the traffic to a VirtualNode associated with the actual service.

At this point, the application is deployed in the mesh but not reachable from outside the cluster because there is no ingress gateway to allow access to users. For this purpose, a pod is deployed with AWS App Mesh envoy proxy image in a deployment resource and exposed using a service of type Load balancer. This attaches a Network load Balancer to the service of the ingress pod, which is then attached to a VirtualGateway to enable route to the VirtualService created for productpage. This establishes the connection between external Load Balancer and internal services, allowing access to end users.

For performing testing scenarios, Fortio is deployed as a pod in namespace Fortio on Node-1. This deployment is exposed with the help of Kube-proxy to

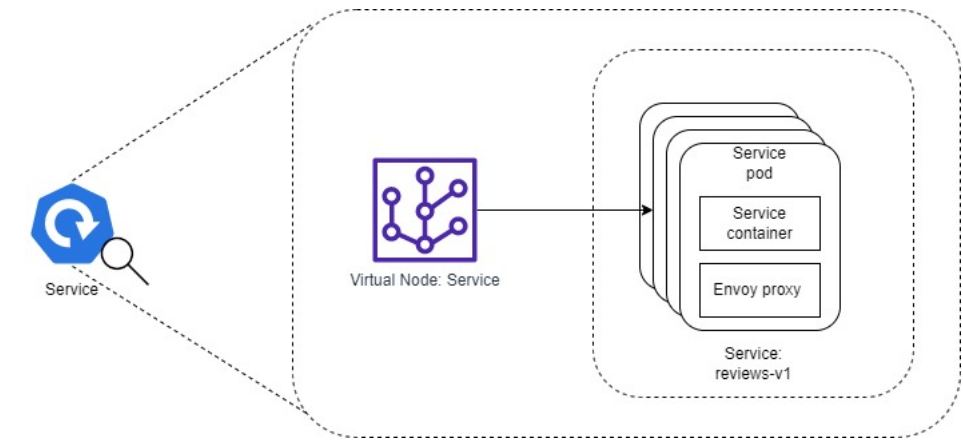


Figure 3.3.2: AWS App Mesh: Service reviews-v1 architecture [10]

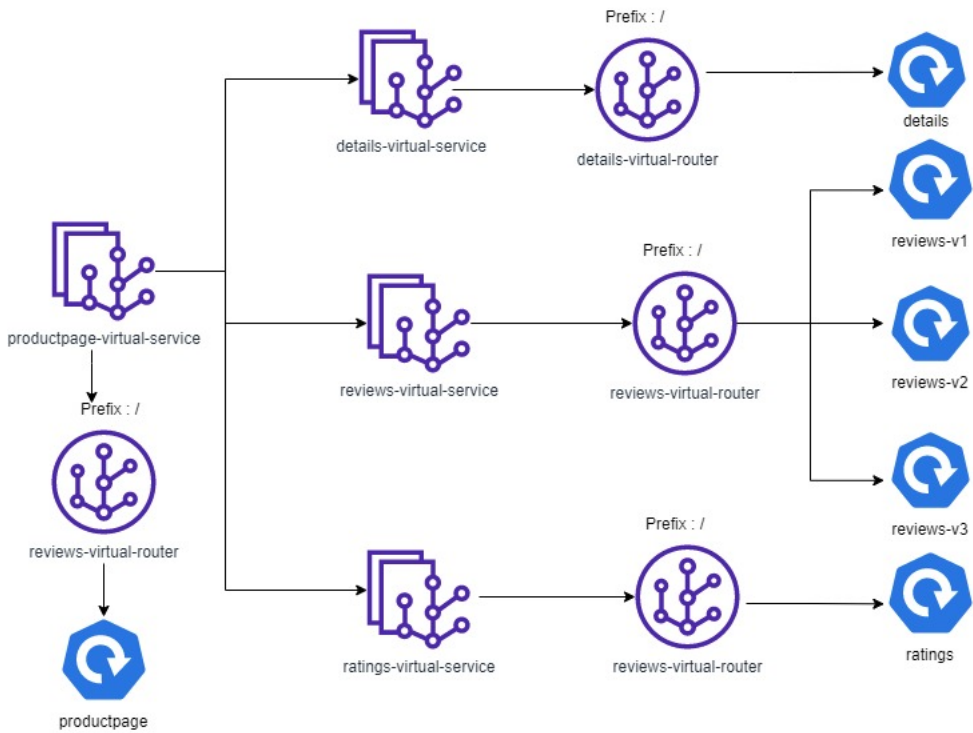


Figure 3.3.3: Application Bookinfo with AWS App Mesh [10]

access it on the local browser. A service can also be created to access Fortio URL from outside the cluster. Scripts were created to conduct automated tests to collect and analyse data. These scripts and their purpose shall be discussed in detail in Section 3.4.

For observability and traceability, Istio offers out-of-box solutions, such as Kiali, Jaeger, Prometheus, and Grafana. In the case of AWS App Mesh, Prometheus and Grafana can be installed with App Mesh-specific dashboards. AWS X-Ray is a service that can be integrated with AWS App Mesh, and used to trace and understand the percentile distribution of response latencies of the application [86]. Kiali and Jaeger are open-source solutions used by Istio to provide the same functionality as the AWS X-ray service. Prometheus, Grafana, Kiali, and Jaeger are deployed in the Istio-system namespace on Node-1. Whereas, AWS X-ray creates a daemon container in each pod in the AWS App-meshed application to collect telemetry data and send it to the AWS X-ray service. To enable AWS X-ray service to collect logs from AWS App Mesh data plane proxies, `tracing.provider=x-ray` and `tracing.enabled=true` parameters are passed to the app-mesh controller helm chart when installing the controller [86].

3.4 Performance testing

After setting up the environment, it is necessary to understand the maximum throughput possible for the service meshed applications. With Fortio's QPS parameter set to 0, it tries to achieve the maximum throughput, but the effect of other parameters such as the number of connections and the number of pods running per service on the throughput is essential to measure [87]. Preliminary tests were executed with different values of QPS, connections, and number of containers with both Istio and AWS App Mesh to understand the relationship between the throughput, the number of concurrent connections made from Fortio, and the number of pods per microservice. It was a common observation in both meshes, that an increasing number of threads results in better throughput, as mentioned in Fortio's GitHub FAQs. As the autoscaling was not set up for pods, it was essential to know the number of pods that should be launched for the scenarios that were conducted. The observations and detailed results are discussed in Chapter 4.

The first scenario is designed to find the highest throughput each meshed application can achieve. A script in Listing 14 is created to run the test 30 times. Each test runs for 30 seconds with QPS set to 0, to achieve the

maximum possible throughput with the provided settings. The number of connections is set to 220 for each test. The reason for selecting 30 seconds for each sample is that time frame was sufficiently large to observe changes in the maximum achieved throughput, choosing a larger number did not make substantial difference in the outcome as the setup was not subjected to any dynamic load changes.

The script in Listing 14 is executed twice, both for the AWS App Meshed application and Istio meshed application. This script generates a histogram of the percentile distribution of response latencies for each test. It also saves the graph and text output in JSON format at a specific path. This is later used for analyzing and comparing collected data.

In the second scenario, 75% of the lowest value of QPS achieved for Istio and AWS App Meshed applications are set as the new QPS, to escape the warnings received during the first scenario testing. To perform this test case, the number of connections is set to 220 and each benchmark is executed for 5 minutes. In total, 30 such samples were collected per meshed application. The test ran for 150 minutes in total for each meshed application.

In the third scenario, a single benchmark is executed with QPS set to 75% of the lowest QPS, connections set to 220, and duration set to 15 minutes. In this experiment, the script is executed for a longer period to stabilize the setup and get more consistent results of CPU utilization and memory consumption for each meshed application.

Details of the collected data values and results are discussed in Chapter 4.

Chapter 4

Results and Analysis

In the pre-benchmarking phase, tests are performed to find the relationship between the achieved QPS, the number of containers, and the number of connections. From the gathered test data in table 4.0.1, we can observe that Istio achieves the best throughput with 10 containers and 98 connections, whereas AWS App Mesh achieves the highest throughput with 10 containers and 220 connections . As per Fortio's FAQ page, the number of connections that can be handled by the mesh is 2 to 4 times the number of CPU cores of the machines. Each node has 32 vCPU cores, so the maximum connections handled per mesh (every meshed application has a dedicated single node) is somewhere between 64 and 128. Although, for AWS App Mesh, this number is almost 6 times the number of CPU cores. If the number of connections is reduced, AWS App Mesh shows lower QPS than it can handle. Therefore in the benchmarking scenarios, 220 is the number of connections used for both Istio and AWS App Mesh. As a general rule, for the best throughput, low latency and a high number of concurrent connections are required, so having a high number of connections is not a problem for Istio. In fact, Istio achieves similar throughput with 98 and 150 connections, as observed in the initial tests. The data listed in Table 4.0.1 is plotted in figure 4.0.1 on a logarithmic scale to make data points easy to compare with each other.

From Figure 4.0.1, we can observe that both the number of containers and the concurrent connections influence the request throughput. However, once the limit is reached, increasing either value does not achieve higher throughput. In the case of Istio, for the test samples 11 and 12, the throughput does not increase with the same number of connections despite scaling out to add 5 more containers per microservice. Samples with a number of connections

Istio				AWS App Mesh			
Requested QPS	Achieved QPS	Number of connections	Number of containers	Requested QPS	Achieved QPS	Number of connections	Number of containers
300	177	32	1	300	293	32	2
300	298	32	2	400	312	32	2
400	311	32	3	400	397	98	5
400	398	32	4	500	496	98	5
500	464	32	4	600	521	32	5
1000	492	32	4	1000	524	32	5
1000	993	64	10	1000	657	64	5
1100	1064	98	10	1000	657	98	5
1500	1000	64	10	1000	625	64	10
1500	1072	98	10	1000	623	98	10
1500	1013	128	10	1100	652	98	10
1500	980	128	15	1000	804	130	10
1500	1000	150	10	1000	907	150	10
No data				1000	925	220	10
				1000	925	250	10
				1033	925	220	10

Table 4.0.1: Initial test data for Istio and AWS App Mesh

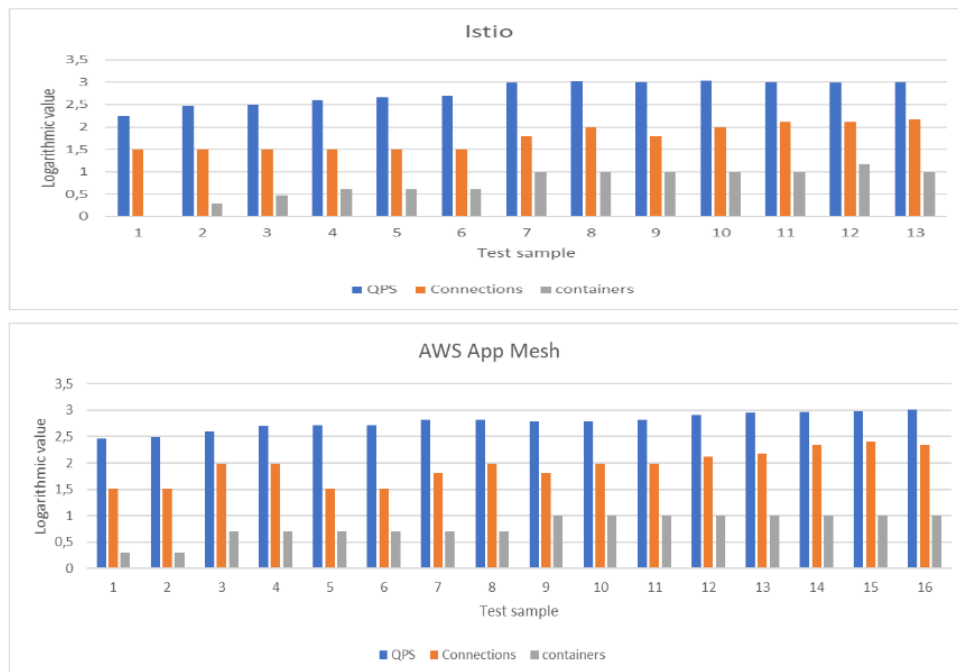


Figure 4.0.1: Istio and AWS App Mesh: Pre-benchmarking test

between 64 and 150 show a similar throughput value of approximately 1000 QPS. For the AWS App Mesh, results can be confirmed with the last 4 test samples, where the throughput seems to have reached the maximum value when the number of connections is 220 and the number of containers is 10. One interesting point to observe is that the AWS App Mesh can handle almost double the number of connections as Istio to get the highest throughput.

After collecting data from the first scenario execution, a script in Listing 15 is created to plot the retrieved data in the form of columns. Original data is in the JSON format, where several details are mentioned. Response latency distribution of p50, p75, p90, p99 and p99.9 is plotted by fetching relevant data from each JSON output. The variable *file_prefix* is changed to get results for both Istio and AWS App Mesh data from Listing 15.

Microsoft Excel is used for plotting the received latencies. Figure 4.0.2 shows that Istio receives throughput between 614 and 1064 QPS. In the case of AWS App Mesh, the throughput ranges from 960 to 1005 QPS. Except for a few values, Istio consistently maintains a throughput above 1000 QPS. For AWS App Mesh, most values are below 1000 QPS with a few values being close to 1005 QPS. There is no drastic difference between the maximum

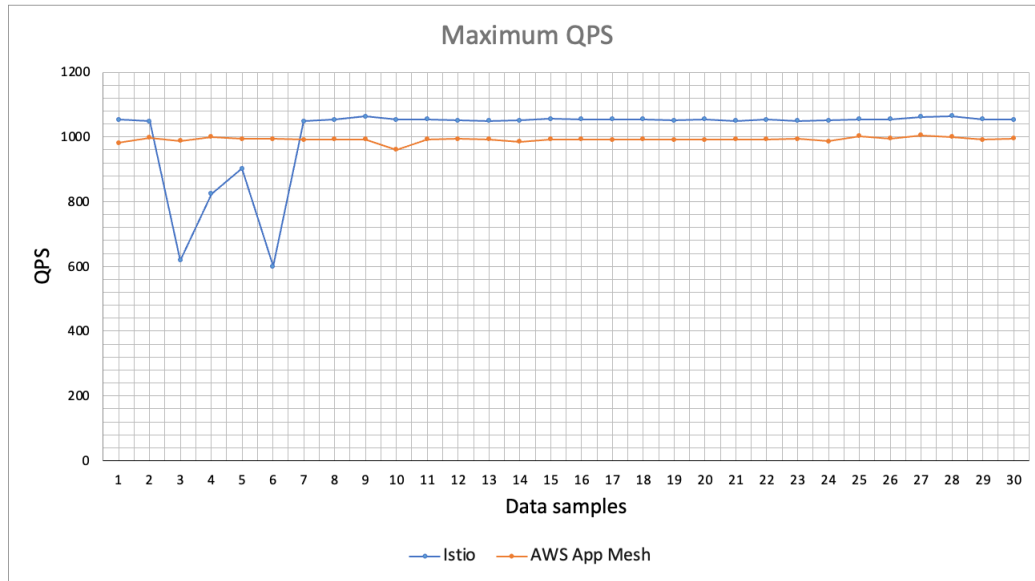


Figure 4.0.2: Maximum QPS supported by Istio and AWS App Mesh

throughput achieved by these meshed applications.

For Istio, even though the achieved throughput is higher than AWS App Mesh by almost 100 QPS, the latency distribution for p99 and p99.9 values are drastically higher than those observed for AWS App Mesh, as shown in Figure 4.0.3 and Figure 4.0.4. The average p99.9 latency for Istio test samples is 1592 ms and for AWS App Mesh, it is 482 ms. The average p99 latency for Istio samples is 1458 ms and for AWS App Meshed samples, it is 422ms. Accepted values for maximum response time without affecting the user experience is 1000 ms for p99 latency. For both p99 and p99.9, this limit is surpassed by Istio. The average p75 latency for Istio is maintained under acceptable limits, which means almost 25% of the requests are exceeding the acceptable latency limit. In the case of AWS App Mesh, both p99 and p99.9 latencies are within the acceptable range. On the other hand, for the average latency (p50), Istio performs better as compared to AWS Mesh. Istio's average of p50 is 118 ms, whereas for AWS App Mesh is 243 ms. It should be noted that this experiment is testing extreme values and the high QPS that each meshed application can handle. This is a border case and may not happen that frequently, as it is the highest QPS handled by the setup. In production scenarios, autoscaling is implemented on both container and virtual machine levels, which ensures that new resources are created to handle the increasing number of requests.

For the second scenario, the lowest value of average QPS is 990 for AWS App Mesh. Therefore, the next test case is run with 75% of 990 QPS, that

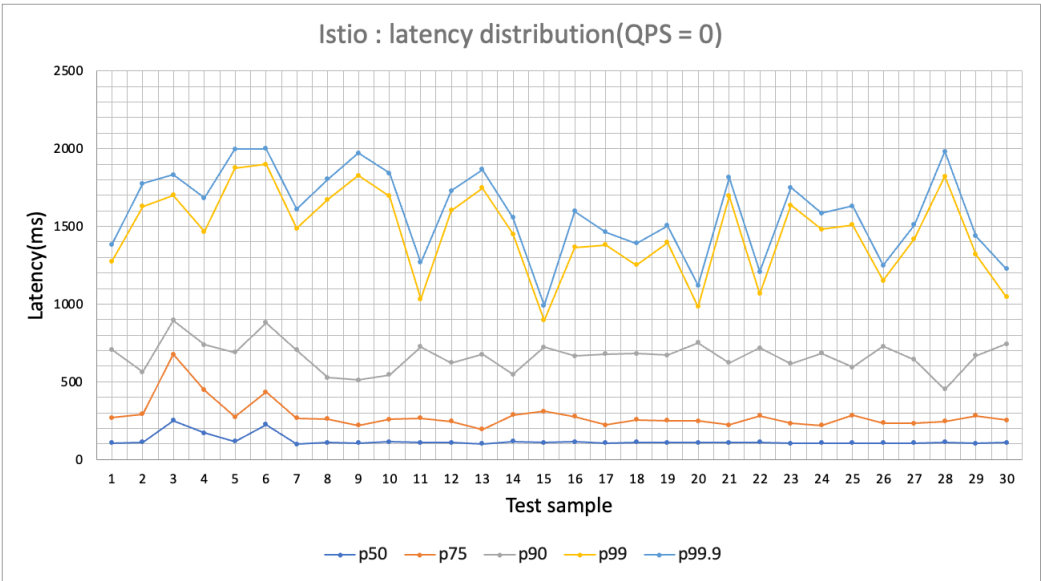


Figure 4.0.3: Istio: Latency distribution graph when QPS is 0

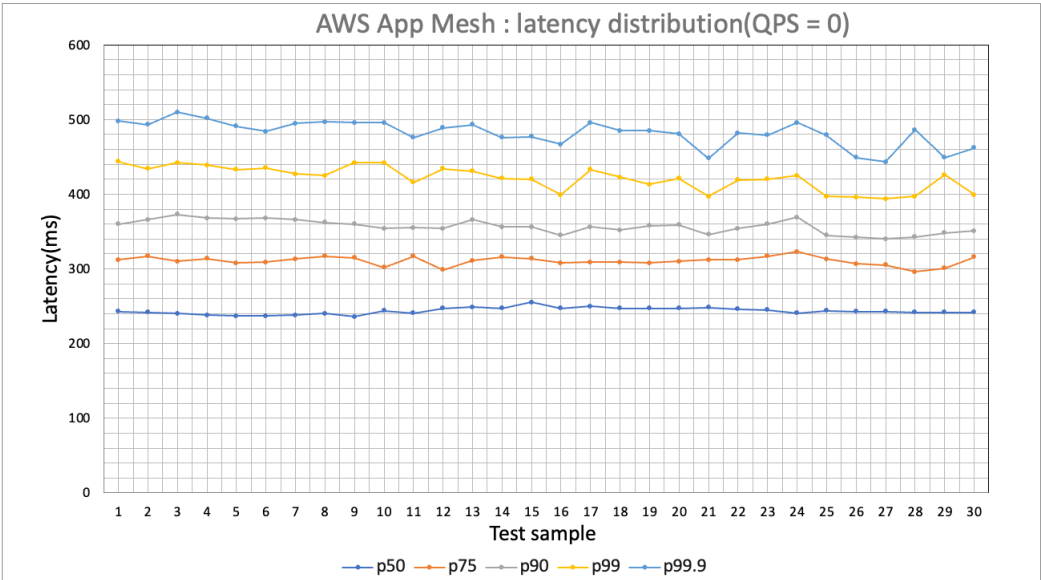


Figure 4.0.4: AWS App Mesh: Latency distribution graph when QPS is 0

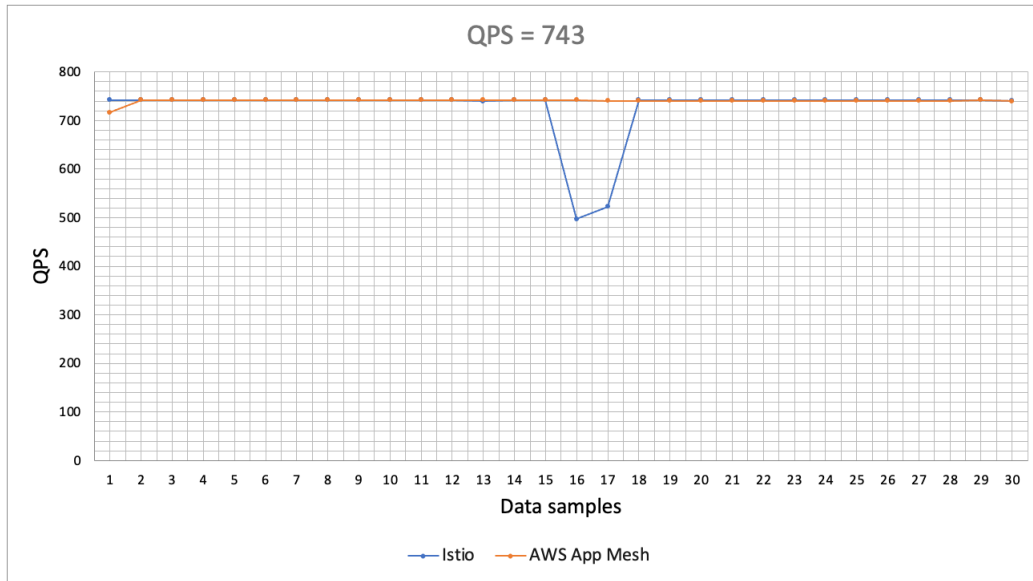


Figure 4.0.5: Actual QPS values supported by Istio and AWS App Mesh when QPS is 743

is, 743 QPS. As the test is run for 5 minutes per test for 30 test samples, the achieved results are more realistic and consistent. From Figure 4.0.5, both the meshes maintained a constant QPS of 743, with 2 tests being an exception in the case of Istio. The achieved QPS fell to almost 500 QPS for these exceptional data points.

The observations in this benchmark are different from the first one. From Figure 4.0.6, in the case of Istio, the p99.9 latency is 288-299 ms, with samples 16 and 17 as exceptions. The two exceptional values of increased latencies seem to have caused lower QPS than expected. In those two cases, p50 and p75 are almost close to 100 ms, which is the lowest latency observed in this test for both Istio and AWS App Mesh. On the contrary, p90, p99, and p99.9 have approximately 800 ms, 1500 ms, and 1800 ms, which are the highest latencies observed in Istio, and have exceeded the acceptable latency limits for p99 and p99.9. Note that, from both the first and second scenarios for Istio-meshed application, if 50% or 75% of the requests complete much faster than the expected latency, in this case (200 ms – 240 ms), the remaining 25% of the requests take much longer time (700-1800 ms) than the expected latency. It appears that the faster response for 75% of the request is compensated by slower responses to 25% of the requests. Whereas, apart from the exceptional cases, the latency distribution is consistent and within the acceptable range.

From Figure 4.0.7, p99.9 lies between 370-593 ms, and p99 lies between

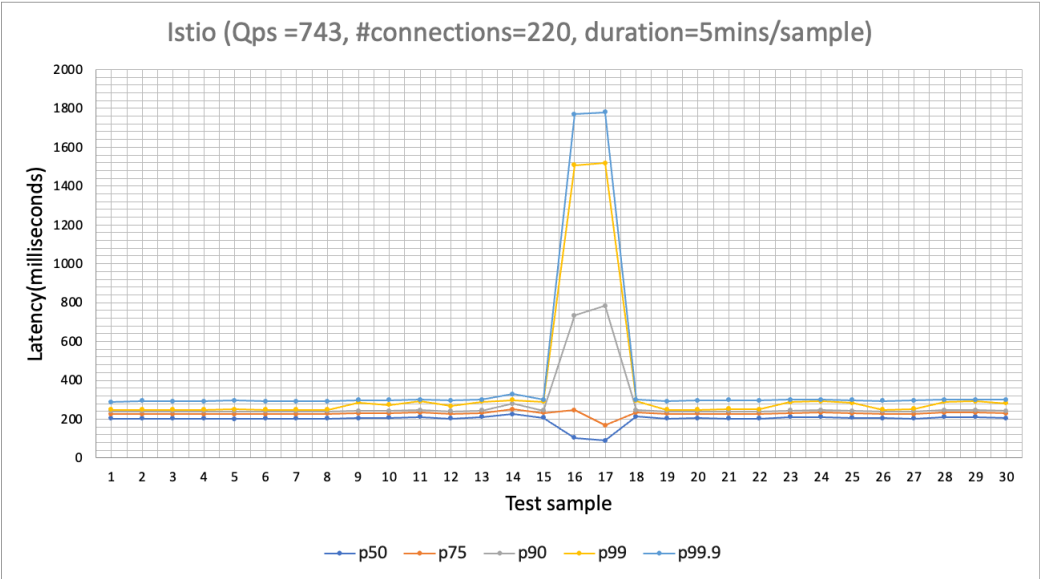


Figure 4.0.6: Istio: Latency when QPS = 743, number of connections = 32 and the test running time is 5 minutes per sample (total 30 samples collected)

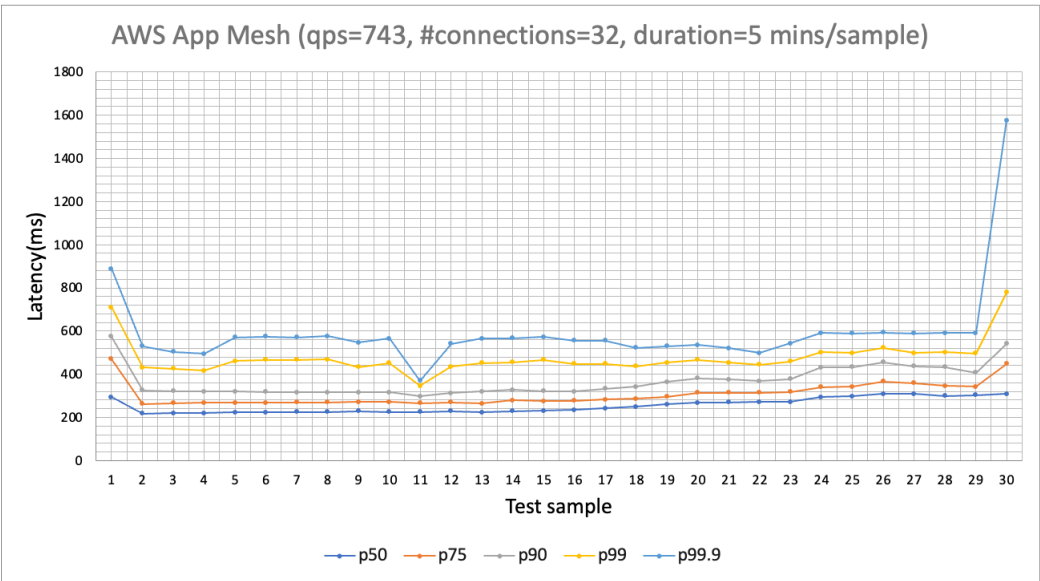


Figure 4.0.7: AWS App Mesh: Latency when QPS = 743, number of connections = 32 and the test is run for 5 minutes per samples (total 30 samples collected)

345-523 ms. Both p99.9 and p99 values are within the acceptable range. Although there are 2 exception data points, samples 1 and 30, where all latency percentiles from p50 to p99.9 have increased than their respective latency ranges. Unlike Istio, where some requests have a faster response than the others in exceptional cases, these data points have overall higher latencies for all requests.

The results from the first scenario suggest that Istio can handle a higher request throughput, although, at the highest possible throughput, the performance suffers significantly for p99 and p99.9 values. Moreover, Istio handles 50-75% of requests with decent response times, which are better than those for AWS App Mesh. AWS App Mesh, on the other hand, performed consistently by keeping the latency percentiles within the acceptable range. In the second scenario, when both the setups are subjected to the same request throughput, AWS App Mesh have overall higher latencies for all percentiles as compared to the respective latency percentiles observed in Istio.

In the third scenario, CPU and memory utilization of both the control plane and data plane is measured for a fair comparison. Overall cluster or node CPU and memory statistics are not taken into consideration, as they do not give proper visibility into the components of the architecture consuming higher resources and causing bottlenecks to the performance. From Figure 4.0.8, it can be seen that the CPU utilization for both setups from the data plane proxies are comparable. For a significant, yet bearable load projected on the meshed applications, the AWS data plane proxy consumes 4-5 vCPU cores, whereas Istio data plane proxies consume 3 to 4 vCPU cores. AWS App Mesh and Istio use 14% and 10% of total available CPU cores, respectively. From Figure 4.0.9, we can observe that for the control plane, the difference between CPU usage in the data plane for both the meshed applications is negligible and therefore difficult to compare. From Figure 4.0.10, we can observe that the memory usage for the data plane in Istio is 0.7% higher than that in AWS App Mesh. For the control plane, the memory usage is again not significant enough for a fair comparison, as shown in figure 4.0.11. Istio consumes only 140MBs more than the AWS App Mesh. A general observation is that AWS App Mesh has more consistency for resource utilization as compared to Istio, as in most graphs, Istio has spiked abruptly when no changes have been explicitly made to the setup.

The control plane resource consumption in the control plane depends on the number of implemented features, as suggested in [50]. As we are testing only a single feature of the canary deployment, it is hard to tell if the results found in this experiment would be the same when more features are added.

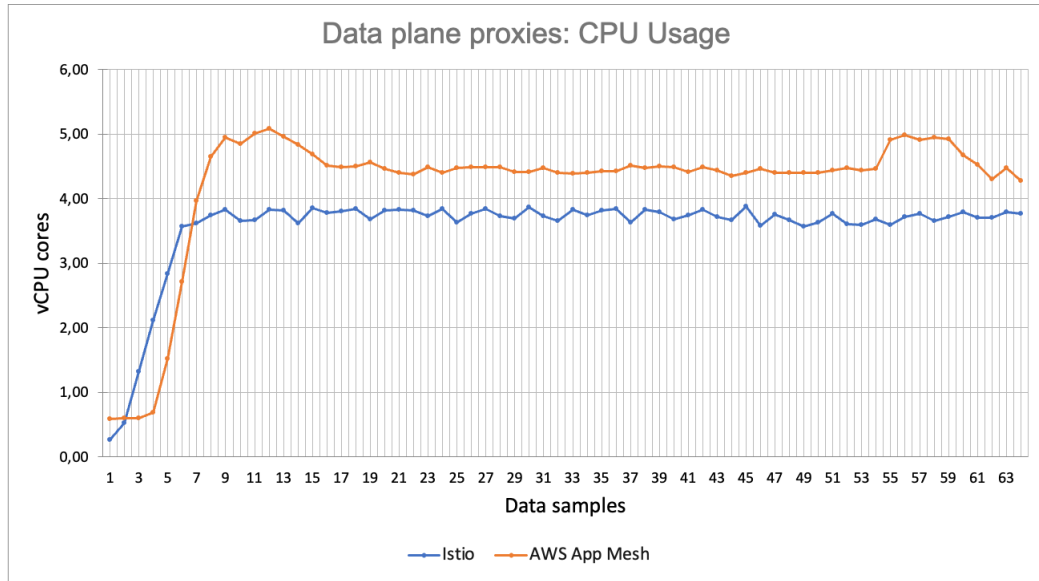


Figure 4.0.8: Data plane: CPU

As far as this experiment is concerned, both AWS App Mesh and Istio are valid choices in terms of CPU and Memory utilization, as there is a negligible difference in resource utilization for both data and control plane.

Looking at the comparison drawn out in Table 4.0.2, Istio proves to be a better solution and in some parameters, AWS App Mesh in others. Therefore, it is not a clear choice to pick one over the other as all the mentioned parameters are deemed equally important for the selection of the mesh. However, performance can be considered an utmost important parameter if we consider deployment efforts and resource utilization secondary. The reason for deployment efforts being secondary is that it is only a different way or require additional steps to achieve the same results in one mesh than the other, but it does not affect the end user in any way. Moreover, resource utilization statistics do not draw out any conclusive results of one mesh being better than the other, therefore, it can also be considered secondary.

In conclusion, AWS App Mesh has more consistent performance in all 3 scenarios. It can handle more concurrent connections as compared to Istio to achieve the maximum possible throughput, and it maintains an acceptable latency range that ensures good QoS and user experience. However, for manageable yet high request load, Istio has performed extremely well and better than AWS App Mesh, with a few exceptions. At Infor, the responsiveness of the application is more vital than consistency, as the customer base can tolerate infrequent lagging response but would expect and

Results and Analysis

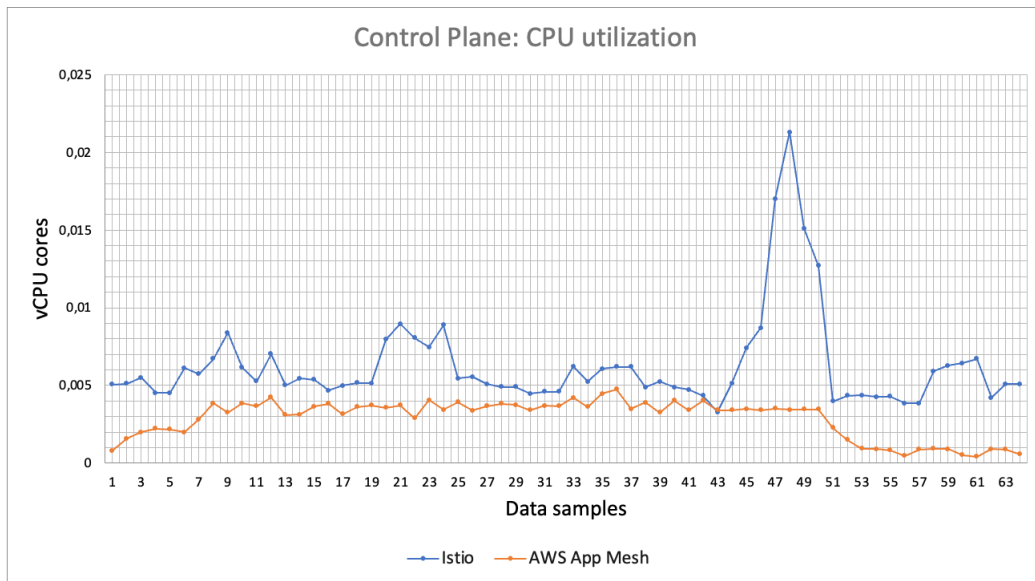


Figure 4.0.9: Control plane: CPU

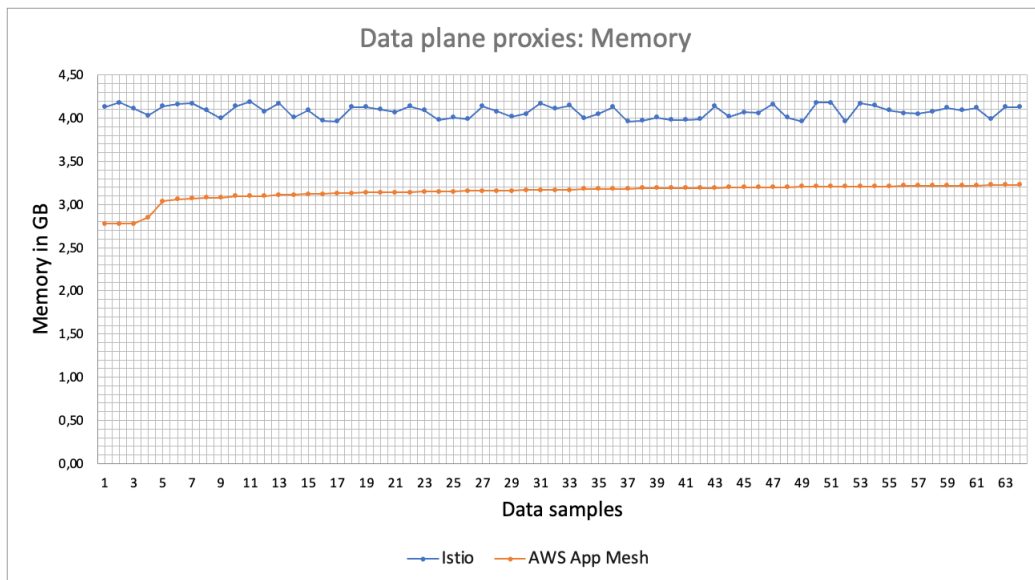


Figure 4.0.10: Data plane: Memory

Results and Analysis

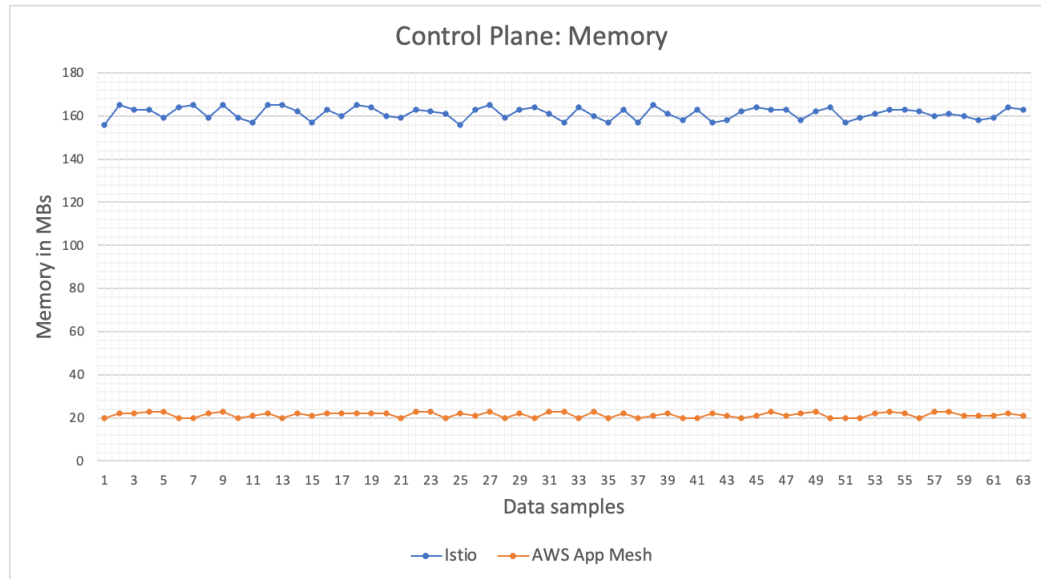


Figure 4.0.11: Control plane: Memory

Compared parameter	Metric	Better solution
Deployment efforts	Number of configurations to achieve end-to-end application connectivity	Istio
API Request	Request throughput	Istio
	Concurrent connections	AWS App Mesh
	Error rate	AWS App Mesh
Latency(max load)	p99.9 / p99 / p90	AWS App Mesh
	p75	Istio / AWS App Mesh
	p50	Istio
Latency(manageable load)	p99.9 / p99 / p90	Istio
	p75	Istio
	p50	Istio
Control plane	Memory usage	AWS App Mesh
	CPU usage	Inconclusive
Data plane	Memory Usage	AWS App Mesh
	CPU Usage	Istio

Table 4.0.2: Comparison based on gathered data and analysis

appreciate extremely fast and responsive application over consistently slower application. Therefore, we can conclude that Istio is a better choice for Infor's production-grade applications, despite the consistency provided by AWS App Mesh in all scenarios.

Although the decision regarding the mesh choice is already clear, it is worth discussing factors such as installation and upgrade process, monitoring support, and overall solution cost to compare the technologies even further. Chapter 5 talks about these factors in detail.

Chapter 5

Discussion

To understand which solution is best suited for Infor's production-grade environments, it is essential to know which candidate satisfies all the parameters that are deemed important from the application perspective. This section will discuss the solutions to compare installation and upgrade processes, deployment efforts, performance, response latencies, throughput, and the cost of the solutions.

Istio is easy to install and has several installation mechanisms, but the AWS AppMesh is easier to install because it has strong ties with AWS services. Both have similar concepts and functionalities. Although, configuring Istio with the bare-minimum features to get the application up and running in the mesh was much easier to configure as compared to AWS App Mesh. This observation is subjective to each individual. However, the number of steps required for configuring Istio to enable applications in the mesh is lower than that for AWS App Mesh.

New versions bring new features or bugfixes to known security or functionality issues. This makes it essential to upgrade the version on a regular basis. Istio is an open-source project, and as such, it has an active community that releases minor versions on a weekly basis. Istio provides different upgrade methods to accommodate an installed version of Istio. These can be tedious to perform without a decent understanding of the working components of Istio. Upgrades are prone to failures in one or more components of Istio, so the process is not recommended for more than two minor versions at a time. Overall, upgrading can be an overwhelming process for Istio, and the method of upgrade depends on the installation method. AWS App Mesh, on the other hand, does not provide any option to upgrade as the service is fully maintained

and managed by AWS.

From the pre-benchmarking experiment, it can be seen that the request throughput is dependent on the number of concurrent connections as well as the number of containers. AWS App Mesh can support a higher number of concurrent connections as compared to Istio, meaning it can sustain a certain throughput while being subjected to a higher number of simultaneous client connections as compared to Istio. When subjected to a high or manageable load, Istio varies the QPS frequently, whereas AWS App Mesh maintains a stable QPS. This means in case of highly volatile load conditions, AWS App Mesh might perform better by providing stable throughput.

In the first benchmarking scenario, Istio can provide higher throughput than AWS App Mesh. Higher throughput signifies serving higher QPS. Meaning, there would be less waiting time for requests to get processed. Moreover, despite 50-75% users getting the best QoS, one downside of Istio is that it spends more time in some requests, and as a result some users might feel the application to be less responsive at certain occasions. In the case of AWS App Mesh, the response time is more evenly distributed, meaning most users will experience an average quality of service. This is a situation where the load is extremely high and it is meant to break the system one way or the other. This scenario is not likely to happen in a fully tested production environment, as it would have auto-scaling and fail-over strategies in place.

The second benchmarking scenario is more believable and achievable in a production-grade environment, where both meshes are subjected to manageable load. In this scenario, Istio maintains an overall very good response time, making QoS excellent for users. However, there are a few samples where the response latencies are beyond acceptable limits, making users starve for a page response for some requests. AWS App Mesh maintains a consistent performance with a good response time, although it is higher than Istio. Users overall see a steady response time with the AWS App Mesh, but with Istio, they notice an even faster response time with except for some cases.

In the third benchmarking scenario, both meshes have very similar values for the CPU and memory usage of the control plane, therefore, it is hard to compare the control planes of both meshes for resource utilisation. For the data plane, Istio data proxies consume fewer resources as compared to AWS App Mesh data proxies. The tests need to be revised and re-performed for the production-like environment to get more significant results. Although, from the performed scenario and achieved result, we can say that both meshes are good options in terms of resource utilization as there is no significant difference in the resource consumption data points collected for both meshes.

As mentioned in the earlier sections, AWS is the chosen cloud technology for the implementation of both Istio and AWS App Mesh solutions. Istio is a free and open-source service mesh, and it does not incur any costs to install it on an existing distributed microservice architecture. However, the underlying infrastructure supporting this architecture is provided by the cloud provider and hence incurs some cost. AWS App Mesh is a service offered by AWS, but it does not incur any additional cost [88]. The only price one needs to pay is for the underlying resources utilized by AWS App Mesh. In both service mesh implementations, EKS is used to provide the underlying architecture with the same node configurations. EKS costs 0.10 USD per hour, resulting in 73 USD per month [89, 90]. An EC2 instance with instance type m5.8xlarge costs 674.52 USD per month, making the total cost 747.52 USD per month. In our performance test scenario, we maintain one EKS cluster, with one node dedicated to AWS App Mesh and another to Istio Mesh. Therefore, the costs remain the same for both Istio and AWS App Meshed application deployments on an EKS cluster with one worker node. However, with AWS App Mesh, the underlying architecture can be modified to support a different type of deployment that is more cost-efficient. EKS supports two types of container launch strategy, EC2 instances and Fargate-based launch [91]. In Fargate, the pricing is calculated from the point of pulling a container image until the termination of the EKS pod, with a minimum one-minute charge applied. As there are no dedicated machines serving as worker nodes in the Fargate solution, no additional costs arise apart from the actual resource utilization by pods. According to the blog post, when workloads are dynamic and only up to a certain threshold, the Fargate solution is more cost-effective regardless of the AWS pricing plan (On Demand, Spot, and Savings)[92]. On the other hand, EC2-launch becomes more cost-effective when workloads utilize the optimal instance capacity.

In the thesis use-case, it can be observed that the workloads do not use more than $(\text{Avg CPU utilization} / \text{Available CPU} * 100 = 6/32 * 100 = 18.75\%)$ 19% CPU of the total available CPU for either Istio-meshed or AWS App Meshed application. As a result, a Fargate-based implementation of this project is a viable option. However, Istio does not support the EKS cluster with Fargate implementation. It has been added to the Istio roadmap since January 2020, although the support has not been added at the time of writing the thesis [93].

Istio and AWS App Mesh can be compared in terms of the monitoring solutions that are built around these technologies. Istio provides an out-of-the-box monitoring solution with Prometheus, Grafana, and Kiali tool stacks for metrics collection, visualization, and traceability, respectively. As these

are open-source tools, no additional cost is required for setting them up on an Istio-enabled cluster. Although, for production-grade monitoring, the data collected from the application needs to be backed up and stored securely. An S3 bucket is a possible storage choice offered by AWS that can be integrated with Istio using the open-source Thanos sidecar [94]. As per the AWS Thanos solution blogpost, the basic components of Thanos ensure data backup and data retrieval to and from S3 buckets, respectively [95]. In this monitoring stack, the user only has to pay for data storage. On the other hand, AWS App Mesh can be integrated with AWS managed Prometheus and Grafana services for data collection and data visualization. Data storage, metric ingestion, and data retrieval incur costs in these monitoring services. Apart from that, AWS X-ray, which provides tracing capabilities for more visibility into applications, also incurs costs for the number of traces recorded, scanned, and retrieved.

Istio solution being mostly open-source requires much less access cost, whereas with AWS App Mesh, one has to pay for every involved service on a usage basis. Unless the open source solution includes cloud-based services such as S3 buckets and load balancers for data storage, an open source solution can be significantly less expensive than an AWS solution. The costs for accessing S3 buckets from open-source implementations can easily surpass the costs of an AWS-supported solution. This can be seen from the sample calculations mentioned in the appendix B. From these calculations, the istio-based solution might actually be costlier than the AWS App Meshed one. However, it is important to do a more accurate calculation based on the actual data requirements and expected loads of Infor's production-grade application.

To summarize, Istio comes out as an adequate service mesh implementation as it provides faster response times as compared to AWS App Mesh for high and manageable load situations in production-grade environments. Resource utilization comparison for both meshes doesn't give a concrete outcome to pick one over the other. In terms of cost, Istio, being an open-source technology, costs less than that of AWS App Mesh for the monitoring solutions built around these technologies. Deployment efforts are subjective and hence can be considered secondary when choosing one technology over the other. For all these reasons, Istio is a better-suited solution than AWS App Mesh for production-grade environments at Infor.

Chapter 6

Conclusions and Future work

6.1 Conclusions

The Thesis project explores into service mesh technology in depth, discussing its architecture and common feature-set. It also takes a deep dive into Istio and AWS App Mesh technologies and compares the two for response latency, throughput, resource utilization, deployment efforts for canary deployment feature, and costs concerning the monitoring solution built around these meshes.

The research mainly explores different scenarios for performance testing of a deployed application. Similar scenarios can be reused for other applications deployed with other service meshes with none or several feature sets. In terms of deployment efforts and performance, Istio stands out by a considerable margin. The production-grade environment is expected to be subjected to a manageable load at most times, where Istio is expected to achieve better response time than AWS App Mesh. Both Istio and AWS App Mesh utilize similar levels of resources. Therefore, resource utilization cannot be considered an influencing factor for the decision of service mesh selection.

The project gives insights into how service mesh technologies work as well as their importance. Service mesh gives more visibility into an application and strengthens the observability to improve monitoring and traceability. It also gives an idea of how comparative analysis works and how to design different scenarios to conduct performance testing of the application.

The thesis project helped to decide which solution is the most suitable to manage production-grade application deployments in Infor's infrastructure.

As mentioned earlier, Infor's infrastructure is highly dependent on the AWS cloud provider. However, in the future, if Infor decides to explore other cloud providers such as GCP or Azure, it is important to reiterate over the selection process of service mesh implementation.

6.2 Limitations

The project gives a service mesh implementation to pursue further, although the feature set required by the application is not fully evolved at the time of implementation. Therefore, canary deployment is the only feature enabled in the meshes while conducting the testing scenarios. With a complete feature set, the performance results may differ for both meshes. Secondly, cost implications need to be further verified with respect to the actual production-grade environment, as the testing environment cannot replicate the request load received in the production environment. Thirdly, autoscaling of both nodes and containers can affect the application performance as well as resource utilization. This project does not explore the impacts of autoscaling on performance. Therefore, it would be an interesting aspect to explore. The results achieved from this thesis are only limited to the selected application, deployment infrastructure, and chosen feature-set of service mesh. It cannot be directly extrapolated to other applications with extra feature-set requirements.

6.3 Future work

Due to the time constraint and the scope limitation of the thesis project, observability and traceability were not explored in-depth during the implementation. The project's future scope may include comparing service meshes to understand their benefits and limitations in terms of observability. This could explore different open-source and cloud-specific monitoring solutions. Furthermore, developers or development operations engineers can be interviewed to understand the necessary feature-set for application deployment and the required metrics for efficient monitoring of the application. The same scenarios designed in the project can be extrapolated and tested with a production-ready application to get more accurate results. In the thesis project, Fortio is used as the benchmarking tool. However, other tools such as locust, JMeter, etc. that can also be further explored to understand their advantages and limitations in the designed scenarios. It might be possible

to achieve more data dimensions or implement different scenarios that may not be plausible with Fortio. Cost analysis can be conducted in a more detailed manner after implementing a production-grade environment to understand the actual cost impacts of service mesh in production.

6.4 Reflections

The project work is important to Infor because it helps the company decide which type of service mesh to investigate further for production-grade environments. Service mesh, being a novel technology, lacks detailed comparative research. This research sets a strong foundation to help researchers understand the importance of service mesh and it proposes a process of selecting the right solution. The research attempts to select a technology that utilizes fewer resources, thereby reducing the cost of the implemented solution and consuming less energy.

Conclusions and Future work

References

- [1] C. Richardson, “What are microservices?” 2021. [Online]. Available: <https://microservices.io/>
- [2] A. Khatri and V. Khatri, *Mastering Service Mesh: Enhance, secure, and observe cloud-native applications with Istio, Linkerd, and Consul*. Packt Publishing Ltd, 2020.
- [3] Mirantis, “Comparison of current service mesh architectures.” [Online]. Available: <https://www.slideshare.net/mirantis/comparison-of-current-service-mesh-architectures>
- [4] Service mesh: Architecture, concepts, and top 4 frameworks. [Online]. Available: <https://www.aquasec.com/cloud-native-academy/container-security/service-mesh/>
- [5] L. Calcote and Z. Butcher, *Istio: Up and running: Using a service mesh to connect, secure, control, and observe*. O’Reilly Media, 2019.
- [6] Istio.io. Architecture. [Online]. Available: <https://istio.io/latest/docs/ops/deployment/architecture/>
- [7] Review: AWS app mesh – a service mesh for EC2, ECS, and EKS | clouonaut. [Online]. Available: <https://clouonaut.io/review-aws-app-mesh-service-mesh-ec2-ecs-eks/>
- [8] AWS app mesh - application-level networking for all your services - amazon web services. [Online]. Available: <https://aws.amazon.com/app-mesh/?aws-app-mesh-blogs.sort-by=item.additionalFields.createdDate&aws-app-mesh-blogs.sort-order=desc&whats-new-cards.sort-by=item.additionalFields.postDateTime&whats-new-cards.sort-order=desc>

REFERENCES

- [9] Istio.io. Bookinfo application. [Online]. Available: <https://istio.io/latest/docs/examples/bookinfo/>
- [10] Managing your EKS traffic with app mesh | by arjen schwarz | DigIO australia | medium. [Online]. Available: <https://medium.com/digio-australia/managing-your-eks-traffic-with-app-mesh-a8838e27e7a1>
- [11] L. Larsson. Benchmarking istio 1.5.0 and linkerd 2.7.1 (master thesis) - elastisys. [Online]. Available: <https://elastisys.com/benchmarking-istio-linkerd-erik-dahlberg-master-thesis/>
- [12] M. Kipper. Benchmarking istio & linkerd CPU. [Online]. Available: https://medium.com/@michael_87395/benchmarking-istio-linkerd-cpu-c36287e32781
- [13] T. Fromm. (2019, May) Kinvolk: Performance benchmark analysis of istio and linkerd. [Online]. Available: <https://kinvolk.io/blog/2019/05/performance-benchmark-analysis-of-istio-and-linkerd/>
- [14] L. De Lauretis, "From monolithic architecture to microservices architecture," in *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 2019. doi: 10.1109/ISSREW.2019.00050 pp. 93–96.
- [15] N. Lei, E. Masanet, and J. Koomey, "Best practices for analyzing the direct energy use of blockchain technology systems: Review and policy recommendations," *Energy Policy*, vol. 156, p. 112422, 09 2021. doi: 10.1016/j.enpol.2021.112422
- [16] "Azure sustainability-sustainable technologies: Microsoft azure." [Online]. Available: <https://azure.microsoft.com/en-us/global-infrastructure/sustainability/#overview>
- [17] "Awsenergy," 1979. [Online]. Available: <https://aws.amazon.com/energy/sustainability/>
- [18] "Microservices pattern: Monolithic architecture pattern," 2021. [Online]. Available: <https://microservices.io/patterns/monolithic.html>
- [19] R. Awati and I. Wigmore, "What is monolithic architecture in software?" May 2022. [Online]. Available: <https://www.techtarget.com/whatis/definition/monolithic-architecture>

REFERENCES

- [20] S. Newman, *Monolith to microservices: evolutionary patterns to transform your monolith*. O'Reilly Media, 2019.
- [21] A. Tsikhanovich, "Monolithic vs. microservices. a quick guide," Jul 2022. [Online]. Available: <https://www.qulix.com/about/monolithic-vs-microservices-architecture/>
- [22] C. Bird, N. Nagappan, B. Murphy, H. Gall, and P. Devanbu, "Don't touch my code! examining the effects of ownership on software quality," 09 2011. doi: 10.1145/2025113.2025119 pp. 4–14.
- [23] R. Posa, *Scala Reactive Programming: Build scalable, functional reactive microservices with Akka, Play, and Lagom*. Packt Publishing, 2018. ISBN 9781787282872. [Online]. Available: <https://books.google.co.in/books?id=VutODwAAQBAJ>
- [24] P. Jausovec, "Fallacies of distributed systems," Nov 2020. [Online]. Available: <https://blogs.oracle.com/developers/post/fallacies-of-distributed-systems>
- [25] A. Kharenko, "Monolithic vs. microservices architecture," Apr 2018. [Online]. Available: <https://articles.microservices.com/monolithic-vs-microservices-architecture-5c4848858f59>
- [26] Q. Zhang, L. Cheng, and R. Boutaba, "Cloud computing: State-of-the-art and research challenges," *Journal of Internet Services and Applications*, vol. 1, no. 1, p. 7–18, 2010. doi: 10.1007/s13174-010-0007-6
- [27] M. Portnoy, *Virtualization essentials*. John Wiley & Sons, 2012, vol. 19.
- [28] G. Sayfan, *Mastering Kubernetes: Master the art of container management by using the power of Kubernetes*. Packt Publishing Ltd, 2018.
- [29] NetApp, "What are containers? – benefits and use cases." [Online]. Available: <https://www.netapp.com/devops-solutions/what-are-containers/>
- [30] S. Baškarada, V. Nguyen, and A. Koronios, "Architecting microservices: Practical opportunities and challenges," *Journal of Computer Information Systems*, vol. 60, no. 5, pp. 428–436,

REFERENCES

2020. doi: 10.1080/08874417.2018.1520056. [Online]. Available: <https://doi.org/10.1080/08874417.2018.1520056>
- [31] J. Turnbull, *The Docker Book: Containerization is the new virtualization*. James Turnbull, 2014.
- [32] D. Bernstein, “Containers and cloud: From lxc to docker to kubernetes,” *IEEE Cloud Computing*, vol. 1, no. 3, pp. 81–84, 2014. doi: 10.1109/MCC.2014.51
- [33] “Container and virtualization tools.” [Online]. Available: <https://linuxcontainers.org/>
- [34] G. Gupta, “Need of container orchestration,” Sep 2019. [Online]. Available: <https://gauravguptacloud.medium.com/need-of-container-orchestration-a9f5dfbee0e3>
- [35] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes: Lessons learned from three container-management systems over a decade,” *Queue*, vol. 14, no. 1, pp. 70–93, 2016.
- [36] “Options for highly available topology,” Jan 2022. [Online]. Available: <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/ha-topology/>
- [37] J. Ghofrani and D. Lübke, “Challenges of microservices architecture: A survey on the state of the practice.” *ZEUS*, vol. 2018, pp. 1–8, 2018.
- [38] Challenges of microservices & when to avoid them – BMC software | blogs. [Online]. Available: <https://www.bmc.com/blogs/microservices-challenges-when-to-avoid/#>
- [39] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, “Service mesh: Challenges, state of the art, and future research opportunities,” in *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2019. doi: 10.1109/SOSE.2019.00026 pp. 122–1225.
- [40] B. Sutter and C. Posta, *Introducing Istio Service Mesh for Microservices*. O’Reilly Media, Incorporated, 2019.
- [41] R. Sharma and A. Singh, *Getting Started with Istio Service Mesh: Manage Microservices in Kubernetes*. Apress, 2020.

REFERENCES

- [42] “Fallacies of distributed computing,” Jul 2022. [Online]. Available: https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing
- [43] R. Chandramouli, Z. Butcher *et al.*, “Building secure microservices-based applications using service-mesh architecture,” *NIST Special Publication*, vol. 800, p. 204A, 2020.
- [44] O. Sheikh, S. Dikaleh, D. Mistry, D. Pape, and C. Felix, “Modernize digital applications with microservices management using the istio service mesh,” in *Proceedings of the 28th Annual International Conference on Computer Science and Software Engineering*, ser. CASCON ’18. USA: IBM Corp., 2018, p. 359–360.
- [45] What’s a service mesh? and why do i need one? [Online]. Available: <https://linkerd.io/2017/04/25/whats-a-service-mesh-and-why-do-i-need-one/>
- [46] (99+) christian posta | LinkedIn. [Online]. Available: <https://www.linkedin.com/in/ceposta/>
- [47] Finagle. [Online]. Available: <https://twitter.github.io/finagle/>
- [48] Netflix/hystrix: Hystrix is a latency and fault tolerance library designed to isolate points of access to remote systems, services and 3rd party libraries, stop cascading failure and enable resilience in complex distributed systems where failure is inevitable. [Online]. Available: <https://github.com/Netflix/Hystrix>
- [49] Comparing service mesh architectures - DZone microservices. [Online]. Available: <https://dzone.com/articles/comparing-service-mesh-architectures>
- [50] Managing microservices with kubernetes and istio | by róža pniewska | kreuzwerker GmbH | medium. [Online]. Available: <https://medium.com/kreuzwerker-gmbh/managing-microservices-with-kubernetes-and-istio-76efea547b28>
- [51] Open Infrastructure Foundation, “Comparing service mesh architectures.” [Online]. Available: <https://www.youtube.com/watch?v=cWfACBIp0a8>
- [52] Service mesh ultimate guide 2020: Managing service-to-service communications. [Online]. Available: <https://www.infoq.com/articles/service-mesh-ultimate-guide/>

REFERENCES

- [53] B. Doerrfeld. 9 open-source service meshes compared. [Online]. Available: <https://techbeacon.com/app-dev-testing/9-open-source-service-meshes-compared>
- [54] servicemesh.es. [Online]. Available: <https://servicemesh.es/>
- [55] AWS fault injection simulator FAQs | fully-managed fault injection service | amazon web services. [Online]. Available: <https://aws.amazon.com/fis/faqs/>
- [56] AWS re:invent 2016: Automating cloud management & deployment (DEV319) - YouTube. [Online]. Available: https://www.youtube.com/watch?v=Epx_32c3c6s
- [57] K8s service mesh: Linkerd or istio | by ohad senior | CloudZone | medium. [Online]. Available: <https://medium.com/cloudzone/k8s-service-mesh-linkerd-or-istio-4bb650d51bc6>
- [58] Istio. [Online]. Available: <https://istio.io/latest/>
- [59] Istio demo: "what's new and exciting in istio 1.8?" - YouTube. [Online]. Available: <https://www.youtube.com/watch?v=pPZNfZLPcZI>
- [60] Meshery. [Online]. Available: <https://layer5.io/cloud-native-management/meshery>
- [61] Gloo mesh enterprise :: Gloo mesh enterprise docs. [Online]. Available: <https://docs.solo.io/gloo-mesh-enterprise/latest/>
- [62] L. Sun and D. Berg, "Istio explained." [Online]. Available: <https://www.oreilly.com/library/view/istio-explained/9781492073963/>
- [63] K. James. (2017, May) Istio: An open microservice mesh for the cloud-native era - wikibon research. [Online]. Available: <https://wikibon.com/istio-an-open-microservice-mesh-for-the-cloud-native-era/>
- [64] Traffic management. [Online]. Available: <https://istio.io/latest/docs/concepts/traffic-management/>
- [65] What is AWS app mesh? - AWS app mesh. [Online]. Available: <https://docs.aws.amazon.com/app-mesh/latest/userguide/what-is-app-mesh.html>

REFERENCES

- [66] Deep. (2020, Dec) Microservices and AWS app mesh. [Online]. Available: <https://medium.com/swlh/microservices-and-aws-app-mesh-f4c7cab9ddca>
- [67] Getting started with AWS app mesh and kubernetes - AWS app mesh. [Online]. Available: <https://docs.aws.amazon.com/app-mesh/latest/userguide/getting-started-kubernetes.html>
- [68] AWS app mesh controller. [Online]. Available: <https://aws.github.io/aws-app-mesh-controller-for-k8s/>
- [69] AWS app mesh features - amazon web services. [Online]. Available: <https://aws.amazon.com/app-mesh/features/>
- [70] R. Alvarez-Parmar and B. Emer. (2020, July) Getting started with AWS app mesh and amazon EKS | containers. Section: Amazon Elastic Kubernetes Service. [Online]. Available: <https://aws.amazon.com/blogs/containers/getting-started-with-app-mesh-and-eks/>
- [71] Private certificate authority - AWS certificate manager - amazon web services (AWS). [Online]. Available: <https://aws.amazon.com/certificate-manager/private-certificate-authority/>
- [72] Benchmarking linkerd and istio. [Online]. Available: <https://linkerd.io/2021/05/27/linkerd-vs-istio-benchmarks/>
- [73] AWS app mesh vs. istio: A comparison of service mesh - vedcraft. [Online]. Available: <https://vedcraft.com/architecture/aws-appmesh-vs-istio-comparison-of-service-mesh/>
- [74] AWS app mesh vs google istio service mesh - bizety. [Online]. Available: <https://www.bizety.com/2019/01/04/aws-app-mesh-vs-google-istio-service-mesh/>
- [75] AWS app mesh vs istio | what are the differences? [Online]. Available: <https://stackshare.io/stackups/aws-app-mesh-vs-istio>
- [76] Understand qps, tps, pv, uv, gm, ip, rps in seconds. [Online]. Available: <https://blog.karatos.in/a?ID=01800-f3813c82-9787-400c-b5ae-adadd7885bb7>
- [77] . M. Read. Installation configuration profiles. [Online]. Available: <https://istio.io/latest/docs/setup/additional-setup/config-profiles/>

REFERENCES

- [78] Kinvolk, “Kinvolk service mesh benchmark suite,” original-date: 2019-05-18T14:00:42Z. [Online]. Available: <https://github.com/kinvolk/service-mesh-benchmark>
- [79] I. Sim, “Linkerd2 benchmark,” original-date: 2018-11-15T05:08:53Z. [Online]. Available: <https://github.com/ihcsim/linkerd2-benchmark/blob/master/gke.sh>
- [80] What is time to first byte, and how to improve it. [Online]. Available: <https://www.searchenginepeople.com/blog/16081-time-to-first-byte-seo.html>
- [81] T. Gupta. Load testing with locust. [Online]. Available: <https://medium.com/swlh/load-testing-with-locust-3e74349f9cbf>
- [82] Best server and application response time monitoring tools + guide - DNSstuff. [Online]. Available: <https://www.dnsstuff.com/response-time-monitoring>
- [83] When to use canary vs. blue/green vs. rolling deployment. [Online]. Available: <https://www.techtarget.com/searchitoperations/answer/When-to-use-canary-vs-blue-green-vs-rolling-deployment>
- [84] A. Joshi. asawari44/IstioVSAppMeshBenchmark: This repository contains code for application deployment and benchmarking for both istio and AppMesh. [Online]. Available: <https://github.com/asawari44/IstioVSAppMeshBenchmark>
- [85] Istio, “Istio/samples/bookinfo at master · istio/istio.” [Online]. Available: <https://github.com/istio/istio/tree/master/samples/bookinfo>
- [86] Integrating AWS x-ray with AWS app mesh | AWS compute blog. [Online]. Available: <https://aws.amazon.com/blogs/compute/integrating-aws-x-ray-with-aws-app-mesh/>
- [87] FAQ · fortio/fortio wiki. [Online]. Available: <https://github.com/fortio/fortio>
- [88] AWS app mesh pricing - amazon web services. [Online]. Available: <https://aws.amazon.com/app-mesh/pricing/>
- [89] Amazon EKS pricing | managed kubernetes service | amazon web services. [Online]. Available: <https://aws.amazon.com/eks/pricing/>

REFERENCES

- [90] AWS pricing calculator. [Online]. Available: <https://calculator.aws/#/estimate>
- [91] Theoretical cost optimization by amazon ECS launch type: Fargate vs EC2 | containers. [Online]. Available: <https://aws.amazon.com/blogs/containers/theoretical-cost-optimization-by-amazon-ecs-launch-type-fargate-vs-ec2/>
- [92] Theoretical cost optimization by amazon ECS launch type: Fargate vs EC2 | containers. Section: Amazon EC2. [Online]. Available: <https://aws.amazon.com/blogs/containers/theoretical-cost-optimization-by-amazon-ecs-launch-type-fargate-vs-ec2/>
- [93] [EKS/fargate] : Support for ISTIO service mesh sidecars · issue #682 · aws/containers-roadmap. [Online]. Available: <https://github.com/aws/containers-roadmap/issues/682>
- [94] Improving HA and long-term storage for prometheus using thanos on EKS with s3 | AWS open source blog. Section: Advanced (300). [Online]. Available: <https://aws.amazon.com/blogs/opensource/improving-ha-and-long-term-storage-for-prometheus-using-thanos-on-eks-with-s3/>
- [95] Improving HA and long-term storage for prometheus using thanos on EKS with s3 | AWS open source blog. Section: Advanced (300). [Online]. Available: <https://aws.amazon.com/blogs/opensource/improving-ha-and-long-term-storage-for-prometheus-using-thanos-on-eks-with-s3/>
- [96] Aeschylus and M. L. West, “Prometheus,” 1992. [Online]. Available: <https://aws.amazon.com/prometheus/pricing/>
- [97] Kiali architecture. [Online]. Available: <https://kiali.io/docs/architecture/architecture/>

REFERENCES

Appendix A

Selection criteria

A discussion about service mesh implementation with a development team at Infor brought up the following points:

1. What technology stack is currently being used in your application architecture?

We are currently using AWS to deploy our microservices on the Elastic Kubernetes Service (EKS) cluster that takes care of container orchestration using Kubernetes. Microservices are deployed with the help of a Continuous Integration/Continuous deployment (CI/CD) pipeline, implemented on GitLab with the help of helm charts. Microservices running inside the cluster are exposed in the Virtual Private Cloud (VPC) with Network LoadBalancer (NLB). We also use Sumologic and Grafana with a telegraph agent for visibility into the application stack.

2. What problems will be solved with the introduction of a service mesh to your architecture?

Currently, authentication to the service stack is completely managed by the customer-facing microservice. The end-user is required to pass a Java Web Token (JWT) authentication token in the request to access the service through the API gateway. This API gateway communicates with a lambda function to check the validity of the token, if the token is valid, the request gets through and the user receives a response. We do not have any way to control inter-service communication. All services are accessible from all other services. Authentication and authorization can be handled more maturely with the help of service

mesh. Secondly, despite having some monitoring in place, it is difficult to get more fine-grained data about microservices such as success rate, request latency, request rate, etc. Canary and Blue-green deployments are part of a roadmap for application deployment production. With the help of service mesh, both observability and routing goals can be achieved as per the service mesh feature set mentioned above. Currently, developers implement retry and timeout logic per microservice to ensure reliability. A service mesh provides a more sophisticated solution such as circuit breaking and rate-limiting to achieve reliability.

3. Is your team ready for this technological shift?

The common opinion is to adopt service mesh as it provides solutions to existing problems and introduces more features. There are other teams in the organization exploring the idea of using a service mesh.

4. What are the requirements or cross-cutting concerns of your application?

Observability, trust, routing, and resilience are the topmost priorities for the application.

Appendix B

Sample cost calculations for service mesh solutions

The following values are based on an example of cost analysis mentioned on AWS official documentation [96].

Assumed parameters

- Data size : 3.34 GB
- Total samples ingested : 892.8 million samples (explained in [96])
- Total Requests made to view dashboard per month: 72000
- Queries sampled per month : 7.2 billion processed queries (This is calculated from the following assumed values, explained in [96]).
 - End users : 1
 - Charts : 20
 - Dashboard viewing time : 2 hours

Solution 1 - Istio-based

- Prometheus : Open-source
- Metric collection : free
- Thanos : No additional network cost if data is traversed in the s3 object store and thanos working in the same aws region
Only object storage and retrieval cost , shall be discussed in s3.
- Grafana : Open-source
- S3 :

Appendix B: Sample cost calculations for service mesh solutions

- Storage : 0.07682 USD ($0.023 \text{ USD per GB} * 3.34$)
- Request and data retrieval: 0.360 USD ($72000/1000 * 0.005 \text{ USD per } 1000 \text{ requests}$)
- kiali - no extra storage needed for kiali. It uses grafana or cluster api or jaeger or prometheus data to retrieve trace info [97]
- Total = around 0.4 USD

Solution 2: AppMesh

- Managed prometheus + managed grafana

- Storage: 0.10 USD ($0.03 \text{ USD per GB} * 3.34 \text{ GB}$)
- Metric ingestion: 80.93 USD ($0.90 \text{ per } 10\text{M samples} * 892.8 \text{ million samples}$)
- Request and data retrieval: \$0.72 ($\$0.10/\text{billion samples processed} * 7.2 \text{ billion samples processed}$)
- Total = 81.75 USD

- AWS X-ray service : Free for first 100000 requests

Appendix C

Listings

```
---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
    - name: v3
      labels:
        version: v3
```

Listing 1: DestinationRule - first approach


```
---
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
    - reviews
  http:
    - route:
        - destination:
            host: reviews
            subset: v1
            weight: 33
        - destination:
            host: reviews
            subset: v2
            weight: 33
        - destination:
            host: reviews
            subset: v3
            weight: 34

---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
    - name: v3
      labels:
        version: v3
```

Listing 2: DestinationRule - second approach

```

apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  selector:
    istio: ingressgateway # use istio default controller
  servers:
  - port:
    number: 80
    name: http
    protocol: HTTP
    hosts:
    - "*"
---
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: bookinfo
spec:
  hosts:
  - "*"
  gateways:
  - bookinfo-gateway
  http:
  - match:
    - uri:
      exact: /productpage
    - uri:
      prefix: /static
    - uri:
      exact: /login
    - uri:
      exact: /logout
    - uri:
      prefix: /api/v1/products
  route:
  - destination:
    host: productpage
    port:
      number: 9080

```

Listing 3: Istio Ingress Gateway

```
---
  apiVersion: appmesh.k8s.aws/v1beta2
kind: Mesh
metadata:
  name: appmesh-bookinfo
spec:
  namespaceSelector:
    matchLabels:
      mesh: appmesh-bookinfo
```

Listing 4: AWS App Mesh creation

```
---
apiVersion: appmesh.k8s.aws/v1beta2
kind: VirtualNode
metadata:
  name: reviews
  namespace: appmesh-bookinfo
spec:
  podSelector:
    matchLabels:
      app: reviews
  listeners:
    - portMapping:
        port: 9080
        protocol: http
  serviceDiscovery:
    dns:
      hostname: reviews.appmesh-bookinfo.svc.cluster.local
```

Listing 5: VirtualNode for service reviews

```
---
apiVersion: appmesh.k8s.aws/v1beta2
kind: VirtualNode
metadata:
  name: reviews-v2
  namespace: appmesh-bookinfo
spec:
  podSelector:
    matchLabels:
      app: reviews-v2
  listeners:
    - portMapping:
        port: 9080
        protocol: http
  serviceDiscovery:
    dns:
      hostname: reviews-v2.appmesh-bookinfo.svc.cluster.local
  backends:
    - virtualService:
        virtualServiceRef:
          name: ratings
```

Listing 6: VirtualNode for service reviews-v2

```
---
apiVersion: appmesh.k8s.aws/v1beta2
kind: VirtualNode
metadata:
  name: reviews-v3
  namespace: appmesh-bookinfo
spec:
  podSelector:
    matchLabels:
      app: reviews-v3
  listeners:
    - portMapping:
        port: 9080
        protocol: http
  serviceDiscovery:
    dns:
      hostname: reviews-v3.appmesh-bookinfo.svc.cluster.local
  backends:
    - virtualService:
        virtualServiceRef:
          name: ratings
```

Listing 7: VirtualNode for service reviews v3

```
---
apiVersion: appmesh.k8s.aws/v1beta2
kind: VirtualService
metadata:
  name: reviews
  namespace: appmesh-bookinfo
spec:
  awsName: reviews
  provider:
    virtualRouter:
      virtualRouterRef:
        name: reviews-virtual-router
```

Listing 8: VirtualService for service reviews

```
---
apiVersion: appmesh.k8s.aws/v1beta2
kind: VirtualRouter
metadata:
  namespace: appmesh-bookinfo
  name: reviews-virtual-router
spec:
  listeners:
    - port mapping:
      port: 9080
      protocol: http
  routes:
    - name: reviews-route
      httpRoute:
        match:
          prefix: /
        action:
          weightedTargets:
            - virtualNodeRef:
              name: reviews
              weight: 1
            - virtualNodeRef:
              name: reviews-v2
              weight: 1
            - virtualNodeRef:
              name: reviews-v3
              weight: 1
```

Listing 9: VirtualRouter for service reviews

```
---
apiVersion: appmesh.k8s.aws/v1beta2
kind: VirtualGateway
metadata:
  name: ingress-gw
  namespace: appmesh-bookinfo
spec:
  namespaceSelector:
    matchLabels:
      gateway: ingress-gw
  gatewayRouteSelector:
    matchLabels:
      gateway: ingress-gw
  podSelector:
    matchLabels:
      app: ingress-gw
  listeners:
    - portMapping:
        port: 8088
        protocol: http
  logging:
    accessLog:
      file:
        path: /dev/stdout
```

Listing 10: Virtualgateway for AWS App Mesh ingress gateway

```
---
apiVersion: appmesh.k8s.aws/v1beta2
kind: GatewayRoute
metadata:
  name: ingress-gw
  namespace: appmesh-bookinfo
spec:
  httpRoute:
    match:
      prefix: "/"
    action:
      target:
        virtualService:
          virtualServiceRef:
            name: productpage
```

Listing 11: gateway route to reach service productpage

```
---
apiVersion: v1
kind: Service
metadata:
  name: ingress-gw
  namespace: appmesh-bookinfo
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 8088
      name: http
  selector:
    app: ingress-gw
```

Listing 12: Service of type loadbalancer exposing ingress gateway


```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ingress-gw
  namespace: appmesh-bookinfo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ingress-gw
  template:
    metadata:
      labels:
        app: ingress-gw
    spec:
      serviceAccountName: appmesh-bookinfo
      securityContext:
        fsGroup: 65534
      containers:
        - name: envoy
          image: 840364872350.dkr.ecr.us-west-2.amazonaws.com/
            aws-appmesh-envoy:v1.17.2.0
          ports:
            - containerPort: 8088
```

Listing 13: AWS App Mesh ingress gateway deployment

```
#!/bin/sh
QPS= 0
connections=220
time=30s
url="<INGRESS_GATEWAY_FOR_ISTIO_OR_AWS_APPMESH>/productpage"
filename_prefix="test"
for i in {1..30}
do
    kubectl exec <fortio_pod_id> -n fortio -- fortio load
    -QPS $QPS -c $connections -t $time -json
    ${filename_prefix}_${i}.json -a $url
    sleep 5
done
```

Listing 14: Bash shell Script to calculate the maximum QPS

```

import requests
def get_QPS(file_prefix):
    for i in range(1,31):
        url = "<FORTIO_URL>/fortio/data/"
        +file_prefix+"_"+str(i)+".json"
        r= requests.get(url)
        json_data = r.json()
        print(str(int(json_data["ActualQPS"]*1000/1000)))
def get_latency(index, file_prefix):
    for i in range(1,31):
        url = "<FORTIO_URL>/fortio/data/"
        +file_prefix+"_"+str(i)+".json"
        r= requests.get(url)
    json_data = r.json()
    print(str(int(json_data
["DurationHistogram"]["Percentiles"]
[index]["Value"]*1000)))
def get_data():
    file_prefix="test"
    print("QPS")
    get_QPS()
    print("p50")
    get_latency(0, file_prefix="test")
    print("p75")
    get_latency(1, file_prefix="test")
    print("p90")
    get_latency(2, file_prefix="test")
    print("p99")
    get_latency(3, file_prefix="test")
    print("p99.9")
    get_latency(4, file_prefix="test")
get_data()

```

Listing 15: Script to format collected data

